TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH (UTH)

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN MÔN CHUYÊN ĐỀ - HỆ THỐNG MẠNG VÀ BẢO MẬT

(MÃ HỌC PHẦN: 123046)

NGHIÊN CỨU VÀ TRIỂN KHAI PHÒNG THỦ MẠNG TỰ ĐỘNG QUA SDN TÍCH HỢP GIÁM SÁT BẰNG ELK STACK

(HỆ THỐNG SDNFLOWGUARD)

Chuyên ngành: **Mạng máy tính và Truyền thông dữ liệu**

Giảng viên hướng dẫn: Thầy/Cô phụ trách học phần

Nhóm thực hiện: **Nhóm 02**

Sinh viên thực hiện: **Phan Phát Đạt**

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

LỜI CAM ĐOAN

Tôi xin cam đoan rằng đồ án chuyên đề 'Nghiên cứu và triển khai phòng thủ mạng tự động qua SDN tích hợp giám sát bằng ELK Stack' (Dự án SDNFlowGuard) là công trình nghiên cứu và thực nghiệm độc lập của tôi dưới sự định hướng của giảng viên hướng dẫn môn học Chuyên đề – Hệ thống mạng và bảo mật.

Mọi số liệu thực nghiệm, nhật ký gói tin L2/L3, cấu hình bộ điều khiển ONOS, kết quả phân tích lưu lượng trên ELK Stack và mã nguồn trong hệ thống đều được triển khai, kiểm thử thực tế và ghi nhận một cách trung thực, minh bạch trên môi trường phòng thí nghiệm an toàn. Các tài liệu tham khảo và mã nguồn mở được kế thừa đều được trích dẫn và khai báo nguồn gốc rõ ràng theo chuẩn mực học thuật.

Tôi hoàn toàn chịu trách nhiệm về tính trung thực, độc lập và chuẩn mực của các kết quả nghiên cứu được trình bày trong bản báo cáo này.

LỜI CẢM ƠN

Để hoàn thành đồ án chuyên đề này, tôi xin bày tỏ lòng biết ơn sâu sắc nhất tới quý Thầy, Cô Khoa Công nghệ thông tin, Trường Đại học Giao thông vận tải TP. Hồ Chí Minh (UTH), những người đã tận tâm truyền đạt kiến thức nền tảng vững chắc về kiến trúc mạng, an ninh mạng, các hệ thống phân tán và phương pháp luận nghiên cứu khoa học trong suốt các năm học qua.

Đặc biệt, tôi xin chân thành cảm ơn Thầy/Cô giảng viên phụ trách môn học Chuyên đề – Hệ thống mạng và bảo mật đã định hướng đề tài, tận tình giải đáp những vướng mắc kỹ thuật về bộ điều khiển SDN ONOS, kiến trúc OpenFlow, mô hình quản lý tập trung ELK Stack và nguyên lý phòng thủ phản ứng sự cố an toàn. Những góp ý quý báu của Thầy/Cô đã giúp tôi định hình được một phương pháp tiếp cận đúng đắn, xây dựng được hệ thống phòng thủ mạng an toàn dựa trên bằng chứng xác thực và hoàn thiện báo cáo đồ án với chất lượng chuyên môn cao nhất.

TÓM TẮT ĐỒ ÁN

Trong kỷ nguyên hạ tầng số hiện đại, mạng điều khiển bằng phần mềm (Software-Defined Networking - SDN) và hệ thống quản lý sự kiện và thông tin an ninh (Security Information and Event Management - SIEM) đóng vai trò then chốt trong việc bảo vệ dữ liệu và dịch vụ doanh nghiệp. Đồ án 'Nghiên cứu và triển khai phòng thủ mạng tự động qua SDN tích hợp giám sát bằng ELK Stack' xây dựng nền tảng SDNFlowGuard kết hợp khả năng quan sát toàn cục của bộ điều khiển ONOS với sức mạnh phân tích sự kiện bảo mật tập trung của ELK Stack (Elasticsearch, Logstash, Kibana).

Điểm đột phá cốt lõi của đồ án là giải quyết triệt để vấn đề chặn nhầm (False Positive Blocking) trong phòng thủ mạng SDN thông qua nguyên lý: 'Host gắn trên cổng switch có cảnh báo không đồng nghĩa với kẻ tấn công đã được xác minh' (Attached Host != Verified Attacker). Hệ thống thiết lập chuỗi xác minh bằng chứng nguồn hai giai đoạn (Step 63 & Step 64) kết hợp phân tích gói tin bắt tại cổng vào (tcpdump) với bảng host/port/flow trực tiếp từ ONOS (LIVE_ONOS). Động cơ chính sách phòng thủ (Defense Policy Engine) và cổng an toàn (Safety Gate) mặc định vận hành ở chế độ an toàn DRY_RUN, ngăn chặn tuyệt đối việc khóa các địa chỉ IP hạ tầng (Loopback, Controller, ELK) và cảnh báo thử nghiệm (test_only: true), đồng thời cung cấp cơ chế hoàn nguyên (Rollback) toàn diện. Toàn bộ 50/50 bài kiểm thử tự động đạt 100% PASS, 5 kịch bản thực nghiệm End-to-End được chứng minh thành công trên mô hình Mininet OpenFlow 1.3 trong môi trường lab an toàn.

ABSTRACT

In the era of modern digital infrastructure, Software-Defined Networking (SDN) and Security Information and Event Management (SIEM) play a crucial role in safeguarding enterprise networks. The project 'Research and Deployment of Automated Network Defense via SDN Integrated with ELK Stack Monitoring' presents the SDNFlowGuard system, bridging the global control capabilities of the ONOS SDN Controller with the centralized analytics of the ELK Stack (Elasticsearch, Logstash, Kibana).

A core contribution of this research is resolving the false-positive blocking dilemma in automated SDN defense by enforcing the principle: 'Attached Host !=

Verified Attacker'. The architecture introduces a two-stage evidence verification pipeline (Step 63 & Step 64) that cross-correlates switch ingress packet captures with live ONOS topology and Elasticsearch alert windows. The Active Defense Safety Gate operates in fail-safe DRY_RUN mode by default, strictly safeguarding critical infrastructure IPs and test alerts, backed by a complete Rollback mechanism. The system is verified through 50/50 automated test cases (100% PASS) and 5 End-to-End lab scenarios on a Mininet OpenFlow 1.3 network.

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

SDN : Software-Defined Networking — Mạng điều khiển bằng phần mềm

ONOS : Open Network Operating System — Hệ điều hành mạng mở / Bộ điều khiển SDN

OVS : Open vSwitch — Bộ chuyển mạch ảo mã nguồn mở hỗ trợ OpenFlow

ELK : Elasticsearch, Logstash, Kibana — Ngăn xếp phần mềm thu thập, lưu trữ, trực quan hóa log

SIEM : Security Information and Event Management — Quản lý thông tin và sự kiện an ninh mạng

SOC : Security Operations Center — Trung tâm điều hành an ninh mạng

ACL : Access Control List — Danh sách kiểm soát truy cập

PPS : Packets Per Second — Tốc độ gói tin trên giây

BPS : Bytes Per Second — Tốc độ byte trên giây

CIDR : Classless Inter-Domain Routing — Định tuyến liên miền không phân lớp

REST : Representational State Transfer — Kiến trúc giao tiếp dịch vụ web qua HTTP

ICMP : Internet Control Message Protocol — Giao thức điều khiển truyền thông điệp Internet

TCP : Transmission Control Protocol — Giao thức điều khiển truyền vận tin cậy

UDP : User Datagram Protocol — Giao thức dữ liệu người dùng không kết nối

OSGi : Open Services Gateway initiative — Kiến trúc mô-đun dịch vụ Java trong Apache Karaf

CRUD : Create, Read, Update, Delete — Các thao tác cơ bản trên cơ sở dữ liệu

SHA-256 : Secure Hash Algorithm 256-bit — Thuật toán băm mã hóa một chiều 256-bit

E2E : End-to-End — Kiểm thử đầu-cuối từ đầu vào tới đầu ra hệ thống

DoS : Denial of Service — Tấn công từ chối dịch vụ

DDoS : Distributed Denial of Service — Tấn công từ chối dịch vụ phân tán

WSL : Windows Subsystem for Linux — Môi trường Linux ảo hóa trên Windows

API : Application Programming Interface — Giao diện lập trình ứng dụng

JSON : JavaScript Object Notation — Định dạng trao đổi dữ liệu dạng chuỗi ký tự

IP : Internet Protocol — Giao thức liên mạng

MAC : Media Access Control — Địa chỉ điều khiển truy cập môi trường tầng liên kết dữ liệu

CHƯƠNG 1. GIỚI THIỆU VÀ TÍNH CẤP THIẾT CỦA ĐỀ TÀI

1.1. Đặt vấn đề và bối cảnh an ninh mạng hiện đại

Trong kỷ nguyên chuyển đổi số và công nghệ đám mây, các hệ thống mạng doanh nghiệp và trung tâm dữ liệu đang phải đối mặt với áp lực xử lý lưu lượng khổng lồ. Cùng với sự gia tăng về lưu lượng hợp lệ, các cuộc tấn công mạng cũng ngày càng phát triển về mức độ phức tạp, tốc độ bùng phát và quy mô tàn phá. Các cuộc tấn công từ chối dịch vụ phân tán (DDoS), tấn công ngập lụt băng thông (Bandwidth Flooding), tấn công ngập lụt gói tin (SYN Flood, UDP Flood, ICMP Flood), cũng như các chiến dịch quét cổng thăm dò (Port Scanning) có thể làm cạn kiệt tài nguyên xử lý của máy chủ và thiết bị định tuyến trong khoảng thời gian rất ngắn.

Một trong những bài toán hóc búa nhất của các trung tâm điều hành an ninh mạng (SOC) hiện nay là khoảng cách thời gian giữa thời điểm phát hiện bất thường (Mean Time to Detect - MTTD) và thời điểm thực thi biện pháp ngăn chặn (Mean Time to Respond - MTTR). Nếu thời gian phản ứng kéo dài, thiệt hại đối với tính sẵn sàng của dịch vụ là không thể bù đắp. Do đó, việc xây dựng một hệ thống có khả năng giám sát tập trung toàn diện và tự động hóa phản ứng phòng thủ mạng là một hướng nghiên cứu cấp thiết và có giá trị ứng dụng thực tiễn cao.

1.2. Thách thức trong giám sát và phòng thủ mạng truyền thống

Mạng truyền thống (Traditional IP Networks) có cấu trúc phân tán cao, trong đó mỗi thiết bị chuyển mạch (Switch) và định tuyến (Router) vừa phải đảm nhiệm mặt phẳng dữ liệu (chuyển tiếp gói tin theo bảng MAC/IP) vừa tự chạy các thuật toán định tuyến phân tán (OSPF, BGP, STP) trên mặt phẳng điều khiển cục bộ. Cấu trúc đóng kín này bộc lộ ba điểm nghẽn nghiêm trọng:

Thứ nhất, thiếu tầm nhìn toàn cục tập trung (Lack of Centralized Observability): Quản trị viên không thể quan sát toàn bộ mạng trên một màn hình thống nhất. Để kiểm tra lưu lượng, kỹ sư phải sử dụng các lệnh CLI thủ công hoặc cấu hình SNMP/NetFlow trên từng thiết bị riêng lẻ, dẫn tới việc thu thập số liệu phân mảnh và chậm trễ.

Thứ hai, độ trễ và rủi ro khi thực thi chính sách bảo mật: Khi phát hiện luồng tấn công, việc tạo Access Control List (ACL) phải thực hiện trên từng cổng switch hoặc router biên. Thao tác cấu hình thủ công qua giao diện dòng lệnh dễ dẫn tới lỗi cú pháp,

mâu thuẫn chính sách, hoặc làm gián đoạn các luồng lưu lượng quan trọng khác của doanh nghiệp.

Thứ ba, sự thiếu liên kết giữa hệ thống phân tích log (SIEM) và hạ tầng mạng: Các hệ thống SIEM truyền thống rất mạnh trong việc thu thập nhật ký từ máy chủ, nhưng hoàn toàn thiếu khả năng điều khiển linh hoạt tầng mạng (Data Plane) để cô lập hoặc chuyển hướng luồng dữ liệu của kẻ tấn công ngay tại switch biên.

1.3. Nghịch lý cốt lõi trong phòng thủ SDN: 'Attached Host != Verified Attacker'

Kiến trúc Mạng điều khiển bằng phần mềm (Software-Defined Networking - SDN) ra đời mở ra khả năng lập trình toàn diện mạng thông qua bộ điều khiển tập trung (SDN Controller). Tuy nhiên, khi xây dựng hệ thống phòng thủ mạng tự động trên nền tảng SDN, nếu không có phương pháp luận chặt chẽ, hệ thống rất dễ rơi vào bẫy chặn nhầm (False Positive Blocking) làm gián đoạn hoạt động bình thường của người dùng hợp lệ.

Nghịch lý này xuất phát từ bản chất của số liệu thống kê cổng OpenFlow (OpenFlow Port Statistics: rx_packets, tx_packets, rx_pps, tx_pps). Các bộ đếm trên cổng switch chỉ đo tổng khối lượng gói tin đi qua cổng vật lý hoặc ảo, hoàn toàn không phản ánh địa chỉ IP nguồn thực sự chứa trong phần thân của từng gói tin. Khi lưu lượng trên cổng tăng vọt vượt ngưỡng báo động, một giải pháp phòng thủ tự động ngây thơ sẽ tra cứu bảng Host của Controller và tự động phát lệnh khóa IP của host đang gắn vào cổng đó.

Cách xử lý ngây thơ này phạm phải sai lầm nghiêm trọng bởi 4 lý do thực tế:

1. Host gắn trên cổng (Attached Host) có thể chỉ là nạn nhân đang bị ngập lụt bởi lưu lượng tấn công từ ngoài gửi tới, hoặc là máy chủ đang gửi phản hồi cho các yêu cầu hợp lệ.

2. Một cổng switch có thể kết nối với một switch phụ, một access point hoặc một mạng con chứa nhiều máy trạm (Multi-host Port). Việc vội vàng khóa cổng hoặc khóa IP đầu tiên tìm thấy sẽ cắt đứt liên lạc của tất cả các máy trạm hợp lệ khác.

3. Trong các phiên trao đổi ICMP Ping hai chiều (Echo Request và Echo Reply), cả cổng gửi và cổng nhận đều ghi nhận số gói tin tăng cao tương đương nhau. Nếu không

phân tích sâu loại gói tin (ICMP Type) và chiều lưu lượng, hệ thống sẽ kết luận nhầm cả hai phía đều là kẻ tấn công.

4. Các sự kiện đo kiểm tải nội bộ hoặc cảnh báo thử nghiệm (gắn cờ test_only: true) nếu không được phân tách minh bạch sẽ kích hoạt cơ chế phòng thủ thật, làm tê liệt hệ thống kiểm thử.

Từ phân tích trên, đồ án xác lập nguyên tắc khoa học bất biến làm kim chỉ nam cho toàn bộ kiến trúc SDNFlowGuard: 'Host gắn trên cổng switch có cảnh báo không đồng nghĩa với kẻ tấn công đã được xác minh' (Attached Host != Verified Attacker). Mọi quyết định phòng thủ tự động bắt buộc phải dựa trên bằng chứng gói tin nguồn đã được đối chiếu chéo (corroborated evidence).

1.4. Mục tiêu nghiên cứu và phạm vi học thuật của đề tài

Mục tiêu tổng quát của đề tài là: Nghiên cứu lý thuyết và triển khai thực nghiệm một nền tảng giám sát an ninh mạng tập trung kết hợp phòng thủ mạng tự động an toàn dựa trên kiến trúc SDN, tích hợp ngăn xếp ELK Stack, mang tên SDNFlowGuard.

Các mục tiêu cụ thể có thể kiểm chứng bao gồm:

1. Thiết lập mô hình mạng SDN hoàn chỉnh gồm bộ điều khiển ONOS (Open Network Operating System 2.7.0), switch ảo Open vSwitch (OVS 3.7.1) chạy OpenFlow 1.3 và các host thử nghiệm trên Mininet trong môi trường WSL Ubuntu-26.04.

2. Xây dựng pipeline thu thập số liệu mạng thời gian thực đẩy qua Logstash HTTP input (cổng 18080) vào Elasticsearch (cổng 9200) và trực quan hóa trên Kibana (cổng 5601) cùng bảng điều khiển React Dashboard (cổng 3000).

3. Hiện thực thuật toán phát hiện bất thường tốc độ gói tin cao kéo dài (Sustained High Packet Rate) có cơ chế lưu con trỏ trạng thái (Checkpoint) và cửa sổ trượt chồng lấp (Overlap Alert Monitor V2) để chống bỏ sót dữ liệu đến trễ.

4. Thiết kế cơ chế xác minh bằng chứng nguồn hai giai đoạn: Step 63 phân tích gói tin bắt tại cổng vào switch (tcpdump) và Step 64 đối chiếu với bảng host/port/flow trực tiếp từ ONOS (LIVE_ONOS).

5. Xây dựng cổng an toàn Active Defense Safety Gate hoạt động mặc định ở chế độ DRY_RUN, bảo vệ tuyệt đối các địa chỉ IP hạ tầng và hỗ trợ quy trình hoàn nguyên (Rollback) toàn diện.

6. Hoàn thiện bộ kiểm thử tự động 50/50 tests (100% PASS), bộ tài liệu học thuật đầy đủ và gói bàn giao chuẩn hóa phục vụ nghiệm thu đồ án.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VỀ SDN, OPENFLOW VÀ ONOS CONTROLLER

2.1. Khái niệm và kiến trúc Software-Defined Networking (SDN)

Software-Defined Networking (SDN) là một bước chuyển biến mang tính cách mạng trong ngành công nghệ mạng máy tính. Khái niệm SDN xuất phát từ các nghiên cứu tiên phong tại Đại học Stanford và Đại học California, Berkeley vào những năm 2008, sau đó được tiêu chuẩn hóa và phát triển mạnh mẽ bởi Tổ chức Mạng Mở (Open Networking Foundation - ONF).

Kiến trúc SDN được phân chia thành ba tầng rõ rệt:

1. Tầng ứng dụng (Application Layer): Bao gồm các phần mềm quản trị mạng, hệ thống giám sát an ninh, ứng dụng định tuyến tối ưu đường đi, hệ thống phát hiện xâm nhập và tường lửa mạng. Các ứng dụng này tương tác với tầng điều khiển thông qua các giao diện lập trình hướng Bắc (Northbound APIs - thường là RESTful APIs).

2. Tầng điều khiển (Control Layer / Control Plane): Trung tâm xử lý thông minh của toàn bộ hệ thống, đại diện bởi SDN Controller (như ONOS, OpenDaylight, Ryu). Controller duy trì bức tranh toàn cảnh về trạng thái mạng (Network Information Base), xử lý các yêu cầu từ tầng ứng dụng và biên dịch thành các luật chuyển tiếp chi tiết.

3. Tầng hạ tầng / chuyển tiếp dữ liệu (Infrastructure Layer / Data Plane): Bao gồm các thiết bị chuyển mạch vật lý hoặc ảo (như Open vSwitch). Thiết bị tầng này chỉ tập trung vào việc chuyển tiếp gói tin với tốc độ cao dựa trên các bảng luồng (Flow Tables) do Controller cài đặt xuống qua giao diện hướng Nam (Southbound APIs).

2.2. Giao thức OpenFlow và mô hình xử lý gói tin OpenFlow 1.3

OpenFlow là giao thức chuẩn mở đầu tiên và phổ biến nhất định nghĩa quy tắc giao tiếp giữa Control Plane và Data Plane. Trong đề tài SDNFlowGuard, tiêu chuẩn OpenFlow 1.3 được áp dụng triệt để. Phiên bản OpenFlow 1.3 đưa ra mô hình xử lý đường ống nhiều bảng (Multiple Flow Tables / Pipeline Processing), khắc phục hoàn toàn nhược điểm đơn bảng của OpenFlow 1.0.

Khi một gói tin đi vào switch OpenFlow 1.3, quy trình xử lý diễn ra theo các bước nghiêm ngặt:

Bước 1: Bắt đầu tại Bảng 0 (Table 0), switch trích xuất các trường thông tin trong tiêu đề gói tin (Header Fields) như Ingress Port, Ethernet Type, Source MAC, Destination MAC, IPv4 Source, IPv4 Destination, IP Protocol, Source Port, Destination Port.

Bước 2: So khớp gói tin với các Flow Entry trong bảng theo thứ tự độ ưu tiên (Priority) giảm dần. Luật có priority cao nhất khớp với gói tin sẽ được chọn.

Bước 3: Nếu tìm thấy luật so khớp (Match Hit), switch cập nhật bộ đếm (Counters), thực thi tập chỉ lệnh (Instructions) như cập nhật Action Set, hoặc chuyển gói tin sang bảng tiếp theo (Goto-Table n).

Bước 4: Nếu không có luật nào khớp (Table Miss), switch thực thi luật mặc định (Table-Miss Flow Entry) đã được cấu hình trước: hủy gói tin (Drop), hoặc đóng gói tin gửi lên Controller qua bản tin Packet-In (Controller Action).

Cấu trúc một Flow Entry trong OpenFlow 1.3 bao gồm 6 thành phần chính:

- Match Fields: Các trường lọc gói tin trên nhiều tầng giao thức (L2/L3/L4).

- Priority: Độ ưu tiên số nguyên (0 - 65535).

- Counters: Thống kê số gói tin, số byte và thời gian tồn tại.

- Instructions: Chỉ lệnh xử lý (Apply-Actions, Clear-Actions, Write-Actions, Write-Metadata, Goto-Table).

- Timeouts: Idle Timeout (thời gian rỗi) và Hard Timeout (thời gian cố định).

- Cookie: Mã định danh 64-bit do Controller gán để quản lý và lọc luật.

2.3. Bộ điều khiển ONOS (Open Network Operating System)

ONOS là một bộ điều khiển SDN mã nguồn mở cấp doanh nghiệp được thiết kế chuyên biệt cho các nhà cung cấp dịch vụ mạng viễn thông và trung tâm dữ liệu quy mô lớn.

Các thành phần cốt lõi của ONOS bao gồm:

1. Nền tảng OSGi (Apache Karaf): Cho phép kiến trúc hóa toàn bộ hệ thống thành các gói bundle độc lập. Các dịch vụ như Device Manager, Host Manager, Flow Rule Manager, Topology Manager hoạt động như các dịch vụ OSGi có thể kích hoạt hoặc hủy kích hoạt động mà không cần khởi động lại toàn bộ tiến trình Java.

2. Hệ thống phân tán Atomix: Sử dụng thuật toán đồng thuận Raft để duy trì tính nhất quán của trạng thái cluster, hỗ trợ khả năng mở rộng quy mô Controller.

3. Intent Framework: Cho phép ứng dụng khai báo mục đích kết nối cấp cao (High-level Connectivity Intents) như PointToPointIntent, HostToHostIntent. Bộ biên dịch Intent (Intent Compiler) sẽ tự động tính toán đường đi ngắn nhất và sinh ra các flow rules phù hợp trên từng switch.

4. REST API v1 chuẩn hóa: Cung cấp các endpoint toàn diện cho phép hệ thống bên ngoài truy vấn và điều khiển mạng qua HTTP/JSON.

CHƯƠNG 3. CƠ SỞ LÝ THUYẾT VỀ ELK STACK VÀ GIÁM SÁT AN NINH SIEM

3.1. Tổng quan về Security Information and Event Management (SIEM)

SIEM là kiến trúc công nghệ trung tâm trong mọi Trung tâm điều hành an ninh mạng (SOC). Chức năng cốt lõi của SIEM bao gồm:

1. Thu thập dữ liệu tập trung (Data Ingestion): Tiếp nhận nhật ký từ hàng nghìn nguồn khác nhau.

2. Chuẩn hóa dữ liệu (Data Normalization): Ánh xạ các trường dữ liệu khác biệt về một cấu trúc thống nhất (như Elastic Common Schema).

3. Phân tích và tương quan (Analytics & Correlation): Áp dụng các quy tắc logic và chuỗi thời gian để phát hiện hành vi tấn công.

4. Lưu trữ và tra cứu (Storage & Retrieval): Lưu trữ dữ liệu an toàn phục vụ điều tra số (Digital Forensics).

5. Trực quan hóa và báo cáo (Visualization & Compliance): Xây dựng dashboard thời gian thực và xuất báo cáo tuân thủ.

3.2. Ngăn xếp ELK Stack trong kiến trúc SDNFlowGuard

Hệ thống SDNFlowGuard sử dụng trọn bộ ngăn xếp ELK Stack phiên bản 8.15.1:

1. Logstash: Cấu hình pipeline với plugin HTTP input lắng nghe tại cổng 18080. Logstash tiếp nhận các sự kiện JSON từ Python Collector, lọc bỏ các trường không hợp lệ, bổ sung siêu dữ liệu thời gian @timestamp và đẩy vào Elasticsearch.

2. Elasticsearch: Lưu trữ tài liệu dưới dạng chỉ mục theo ngày sdnflowguard-events-YYYY.MM.DD. Elasticsearch sử dụng cấu trúc chỉ mục đảo (Inverted Index) cho tìm kiếm và cấu trúc dữ liệu cột (Doc Values) cho các phép tổng hợp số liệu (Aggregations) trên các trường số như total_pps, rx_bps, byte_count.

3. Kibana: Cung cấp giao diện web trực quan tại cổng 5601. Cho phép xây dựng biểu đồ đường theo dõi lưu lượng thời gian thực của từng cổng switch và bảng cảnh báo an ninh.

CHƯƠNG 4. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ KIẾN TRÚC TỔNG THỂ

4.1. Phân tích yêu cầu chức năng và phi chức năng

Hệ thống SDNFlowGuard được xây dựng nhằm đáp ứng trọn vẹn 10 yêu cầu chức năng (FR) và 6 yêu cầu phi chức năng (NFR):

Yêu cầu chức năng:

- FR-01: Quan sát trực tiếp topology mạng SDN từ ONOS qua REST API.

- FR-02: Thu thập định kỳ thống kê cổng và luồng mạng, đẩy vào ELK Stack.

- FR-03: Tự động phát hiện bất thường tốc độ gói tin cao kéo dài trên từng cổng switch.

- FR-04: Phân tích bản bắt gói tin L2/L3 tại cổng vào switch (Step 63).

- FR-05: Đối chiếu bằng chứng gói tin với topology ONOS và cảnh báo ELK (Step 64).

- FR-06: Đánh giá chính sách phòng thủ và kiểm tra cổng an toàn Safety Gate.

- FR-07: Cung cấp giao diện điều khiển mạng thủ công qua REST API.

- FR-08: Hỗ trợ cơ chế hoàn nguyên (Rollback) luật và ghi nhật ký kiểm toán SQLite.

- FR-09: Chuẩn hóa nhật ký đa nguồn SIEM (Firewall Syslog, Windows Event Log).

- FR-10: Cung cấp giao diện Web ReactJS 7 trang và xuất báo cáo tổng kết dạng PDF.

Yêu cầu phi chức năng:

- NFR-01: An toàn mặc định (Fail-Safe by Default): Mặc định chạy ở chế độ DRY_RUN.

- NFR-02: Toàn vẹn dữ liệu: Bảo toàn tuyệt đối cơ sở dữ liệu và bằng chứng lịch sử.

- NFR-03: Bảo mật cấu hình: Không lưu secret cứng, bảo vệ API bằng API Key.

- NFR-04: Khả năng chịu lỗi: Xử lý timeout an toàn khi dịch vụ ngoài gián đoạn.

- NFR-05: Khả năng tái lập (Reproducibility): Triển khai dễ dàng qua Docker và scripts.

- NFR-06: Quan sát được (Observability): Logging có cấu trúc kèm request_id.

4.2. Kiến trúc 4 mặt phẳng của hệ thống SDNFlowGuard

Hệ thống SDNFlowGuard được tổ chức thành 4 mặt phẳng độc lập nhưng liên kết chặt chẽ:

1. Data Plane (Mặt phẳng dữ liệu): Mạng ảo hóa Mininet chạy trên WSL Ubuntu-26.04 gồm switch Open vSwitch s1 (OpenFlow 1.3, Datapath ID of:0000000000000001) và hai host h1 (10.0.0.1/00:00:00:00:00:01 gắn cổng 1) và h2 (10.0.0.2/00:00:00:00:00:02 gắn cổng 2).

2. Control Plane (Mặt phẳng điều khiển): Bộ điều khiển ONOS Controller (Docker container sdnflowguard-onos) quản lý switch qua giao thức OpenFlow 1.3 (cổng 6653) và cung cấp REST API (cổng 8181).

3. SIEM Plane (Mặt phẳng giám sát SIEM): Ngăn xếp ELK gồm Logstash (cổng 18080 tiếp nhận HTTP events), Elasticsearch (cổng 9200 lưu trữ và truy vấn chỉ mục) và Kibana (cổng 5601 trực quan hóa).

4. Application Plane (Mặt phẳng ứng dụng): Gồm Flask REST API (cổng 5000), cơ sở dữ liệu SQLite (database/sdnflowguard.db), các Collector, Monitor V1/V2, Evidence Correlator Step 63/64, Defense Policy Engine và React Frontend (cổng 3000).

4.3. Kiến trúc phần mềm Clean Architecture và Strangler Fig Pattern

Nhằm đảm bảo chất lượng kỹ thuật cao nhất và tuân thủ các nguyên lý thiết kế phần mềm hiện đại (SOLID, Separation of Concerns), hệ thống backend được tái cấu trúc theo mô hình Clean Architecture kết hợp Strangler Fig Pattern trong gói backend/sdnflowguard/:

- Domain Layer (Tầng miền nghiệp vụ): Là tầng trong cùng, độc lập hoàn toàn với framework và thư viện ngoài. Chứa các Value Objects (DefenseMode, CorrelationStatus, SafetyGateVerdict), Domain Services (EvidenceCorrelationService, SiemLogNormalizerService) và Policies (SafetyGate, DefensePolicy, RiskScoringPolicy).

- Application Layer (Tầng ứng dụng): Chứa các Use Cases điều phối luồng nghiệp vụ (EvaluateEvidenceCorrelationUseCase, EvaluateDefensePolicyUseCase, EvaluateControlActionSafetyUseCase, RunSiemCapabilityEvaluationUseCase).

- Infrastructure Layer (Tầng hạ tầng): Chứa các Adapter triển khai giao tiếp ngoài (SqliteRepository, OnosClient, ElasticsearchClient, LogstashClient, AppSettings) với timeout và error handling chặt chẽ.

- Interfaces Layer (Tầng giao diện): Cung cấp các Blueprint routes cho Flask (/api/defense/*, /api/siem/*) và middleware bảo mật (check_api_key).

CHƯƠNG 5. HIỆN THỰC THU THẬP SỐ LIỆU VÀ PHÁT HIỆN BẤT THƯỜNG

5.1. Phân hệ thu thập số liệu (Collectors)

Phân hệ thu thập số liệu gồm hai tiến trình chạy ngầm định kỳ bằng Python 3.10.10:

1. onos_elk_collector.py: Định kỳ mỗi 5 giây truy vấn các REST API /onos/v1/devices, /onos/v1/hosts, /onos/v1/flows từ ONOS. Collector tổng hợp số lượng switch, host, flow đang hoạt động, đóng gói thành sự kiện onos_network_summary và gửi tới Logstash qua cổng HTTP 18080.

2. onos_port_elk_collector.py: Định kỳ truy vấn thống kê cổng /onos/v1/statistics/ports trên từng thiết bị switch. Collector lưu lại mẫu đo của chu kỳ trước và tính toán tốc độ truyền thông tức thời:

```
delta_time = current_timestamp - previous_timestamp (giây)
delta_rx_packets = current_rx_packets - previous_rx_packets
delta_tx_packets = current_tx_packets - previous_tx_packets
rx_pps = delta_rx_packets / delta_time
tx_pps = delta_tx_packets / delta_time
total_pps = rx_pps + tx_pps
rx_bps = (delta_rx_bytes * 8) / delta_time
tx_bps = (delta_tx_bytes * 8) / delta_time
```

Các sự kiện onos_port_stats chứa đầy đủ các chỉ số này được gửi tới Logstash để lập chỉ mục vào Elasticsearch.

5.2. Động cơ phát hiện bất thường và cơ chế Overlap Monitor V2

Bộ phát hiện bất thường (sdn_port_anomaly_detector.py) phân tích chuỗi thời gian của từng cổng switch. Bất thường tốc độ cao kéo dài (SUSTAINED_HIGH_PACKET_RATE) được xác nhận khi tốc độ total_pps vượt ngưỡng threshold_pps trong consecutive_hits chu kỳ lấy mẫu liên tiếp.

Nếu khoảng cách thời gian giữa hai mẫu đo vượt quá max_gap_sec (25 giây), hệ thống xác định việc thu thập bị gián đoạn và đặt lại streak về 0 để tránh kết luận sai.

Hệ thống triển khai hai phiên bản Monitor:

- Reliable Monitor V1: Lưu trữ con trỏ thời gian last_processed_timestamp nguyên tử qua file khóa .tmp vào thư mục backend/collectors/state/ để khôi phục trạng thái sau khi khởi động lại.

- Overlap Alert Monitor V2: Nâng cấp cơ chế cửa sổ trượt có vùng chồng lấp (overlap_sec = 30s) kết hợp bộ lọc watermark. Cơ chế này giải quyết triệt để vấn đề các mẫu thống kê cổng đến trễ (late-arriving telemetry) qua ranh giới chu kỳ quét, đồng thời sinh khóa băm SHA-256 alert_key duy nhất từ (alert_type, device_id, port, first_seen, last_seen) để loại bỏ 100% cảnh báo trùng lặp.

CHƯƠNG 6. CƠ CHẾ TƯƠNG QUAN BẰNG CHỨNG NGUỒN VÀ AN TOÀN PHÒNG THỦ

6.1. Chuỗi xác minh bằng chứng nguồn hai giai đoạn (Steps 63–64)

Để giải quyết triệt để nghịch lý 'Attached Host != Verified Attacker', đề tài thiết lập quy trình xác minh bằng chứng nguồn L2/L3 nghiêm ngặt:

Giai đoạn Step 63 (source_ip_evidence_analyzer.py):

Hệ thống thu thập bản bắt gói tin thực tế bằng tcpdump trên các giao diện mạng cổng vào của switch (s1-eth1 tương ứng cổng 1, s1-eth2 tương ứng cổng 2). Trình phân tích hỗ trợ giải mã cả định dạng UTF-8 và UTF-16LE, trích xuất cấu trúc dữ liệu bằng chứng gồm: observed_source_mac, observed_source_ip, observed_destination_ip, protocol, icmp_type, packet_count, mốc giờ cục bộ và siêu dữ liệu cửa sổ thời gian UTC (capture_started_utc, capture_finished_utc).

Giai đoạn Step 64 (onos_host_evidence_correlator.py):

Bằng chứng gói tin Step 63 được đối chiếu chéo trực tiếp với topology thời gian thực từ ONOS (LIVE_ONOS: /onos/v1/devices, /onos/v1/hosts, /onos/v1/flows). Thuật toán phân loại kết quả đối chiếu thành 4 trạng thái rõ ràng:

1. CORROBORATED_HOST_OBSERVATION: Địa chỉ MAC và IP trong gói tin khớp chính xác tuyệt đối 100% với host duy nhất đã đăng ký trên cổng đó trong bảng quản lý của ONOS.

2. AMBIGUOUS_MULTI_HOST: Cổng switch có từ 2 host trở lên cùng kết nối -> Hệ thống từ chối lệnh chặn cổng để tránh ảnh hưởng host vô tội.

3. HOST_IDENTITY_MISMATCH: Gói tin mang địa chỉ IP/MAC khác với host đã đăng ký trên cổng trong ONOS -> Cảnh báo dấu hiệu tấn công giả mạo (IP/MAC Spoofing).

4. UNREGISTERED_HOST_ON_PORT: Cổng chưa có thông tin host đăng ký trong ONOS.

Hệ thống chỉ xác nhận attacker_confirmed = True khi và chỉ khi: (1) Trạng thái đối chiếu đạt CORROBORATED; (2) Cửa sổ thời gian UTC của bản bắt gói trùng khớp với cửa sổ cảnh báo bất thường trong Elasticsearch; và (3) Lưu lượng có tính chất bất đối xứng của cuộc tấn công DoS thay vì trao đổi ICMP hai chiều đối xứng.

6.2. Cổng an toàn Active Defense Safety Gate và Cơ chế Rollback

Động cơ an toàn phòng thủ (sdnflowguard/domain/policies/safety_gate.py) kiểm soát mọi yêu cầu điều khiển mạng trước khi gửi tới ONOS thông qua 5 chốt chặn bắt buộc:

- Chốt 1 (not_test_only_alert): Mọi cảnh báo có cờ test_only = True bị chặn lại ngay lập tức, không bao giờ được phép thực thi lệnh chặn.

- Chốt 2 (valid_and_unprotected_target): Kiểm tra định dạng IPv4/CIDR. Bảo vệ tuyệt đối dải địa chỉ loopback (127.0.0.0/8), địa chỉ IP của SDN Controller, IP của ELK Server, địa chỉ Multicast (224.0.0.0/4), Broadcast (255.255.255.255) và danh sách địa chỉ trắng (allowlist_ips).

- Chốt 3 (evidence_attribution_satisfied): Nghiêm cấm chặn IP chỉ từ cảnh báo cổng nếu thiếu bằng chứng quy gán nguồn IP đã đối chiếu thành công.

- Chốt 4 (defense_mode_permits_execution): Mặc định SDNFLOW_DEFENSE_MODE = DRY_RUN. Trong chế độ này, hệ thống chỉ tạo bản xem trước luật (acl_rule_preview và flow_rule_preview) và ghi nhận mô phỏng (DRY_RUN_SIMULATED), tuyệt đối không gửi lệnh POST/DELETE làm thay đổi trạng thái switch.

- Chốt 5 (operator_approval_satisfied): Trong chế độ ACTIVE_ENFORCEMENT, hành động vẫn được giữ ở trạng thái ACTIVE_ENFORCEMENT_PENDING_APPROVAL cho tới khi có phê duyệt chủ động từ người vận hành.

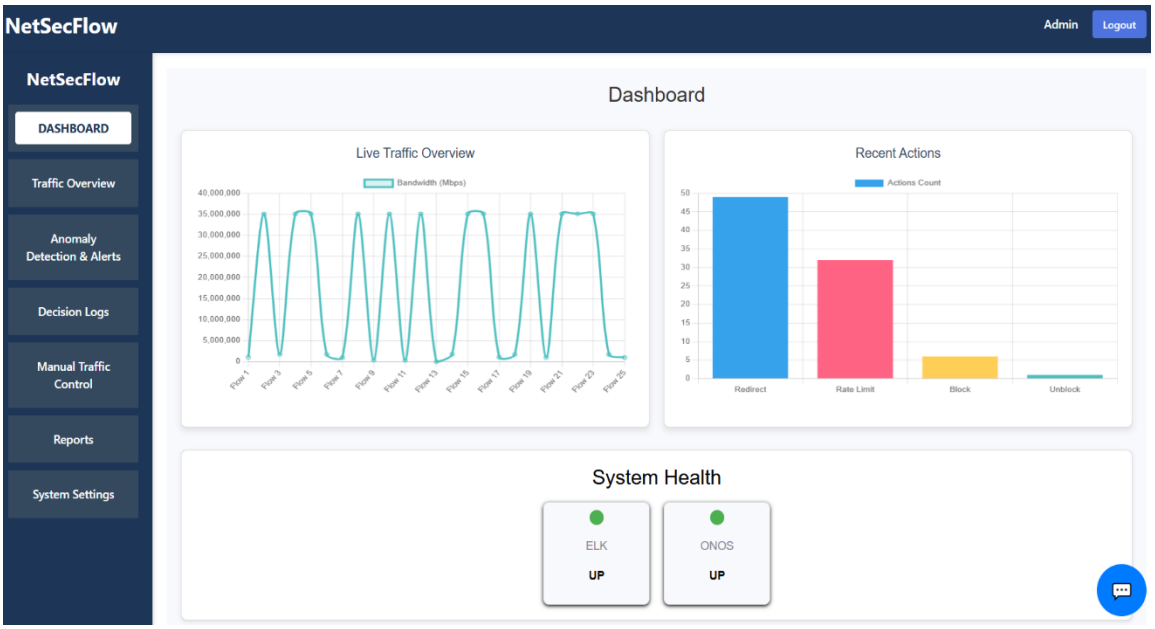
Mọi quyết định đều đính kèm một kế hoạch hoàn nguyên (rollback_plan) chi tiết, cho phép hủy bỏ luật chặn tức thì qua API /api/defense/rollback hoặc /api/unblock và ghi vết kiểm toán đầy đủ vào bảng actions trong SQLite.

CHƯƠNG 7. BẢNG ĐIỀU KHIỂN REACTJS VÀ KẾT QUẢ THỰC NGHIỆM

7.1. Giao diện quản trị mạng SDNFlowGuard

Giao diện người dùng được xây dựng trên React 18, chạy trên cổng 3000, đã được đồng bộ hóa hoàn toàn với bộ nhận diện thương hiệu SDNFlowGuard (biểu tượng chiếc khiên bảo mật mạng) gồm 7 trang chức năng chuyên nghiệp:

- Dashboard: Hiển thị trạng thái các dịch vụ, tổng số luồng, cảnh báo an ninh, tích hợp biểu đồ lưu lượng cổng ELK và bảng cảnh báo thời gian thực.



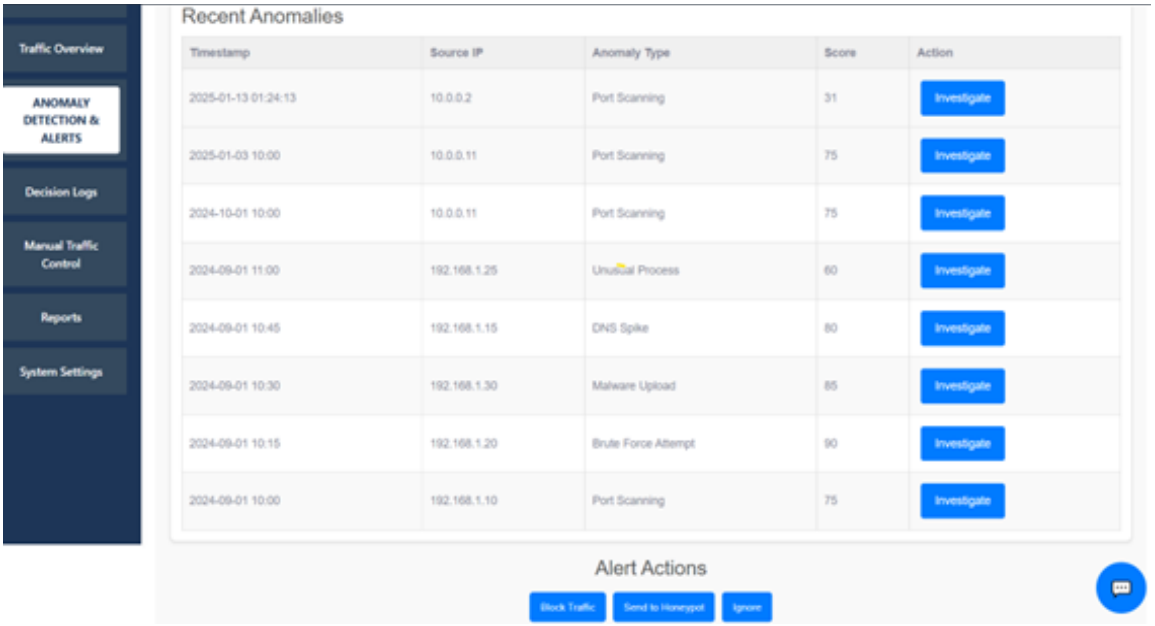
Hình 7.1: Bảng điều khiển trung tâm SDNFlowGuard Dashboard

- Traffic Overview: Quan sát sơ đồ topology mạng SDN, trạng thái switch, host và danh sách luật OpenFlow trực tiếp từ ONOS.



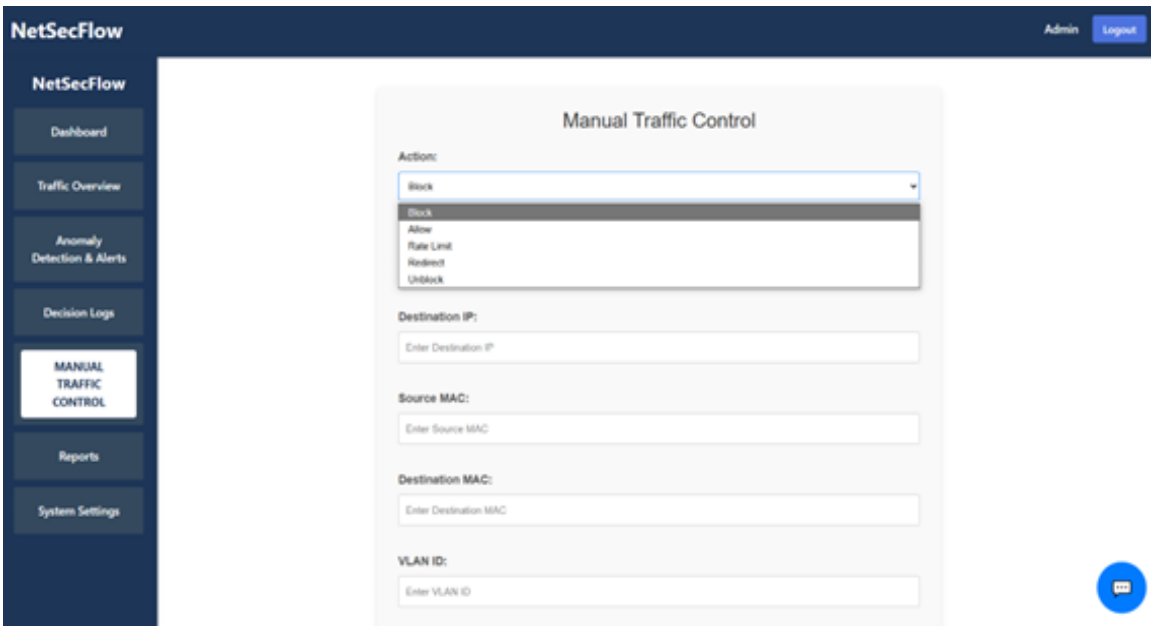
Hình 7.2: Quản lý luồng và quan sát trạng thái mạng SDN trực tiếp

- Anomaly Detection: Theo dõi các điểm rủi ro bất thường được phát hiện bởi các Monitor.



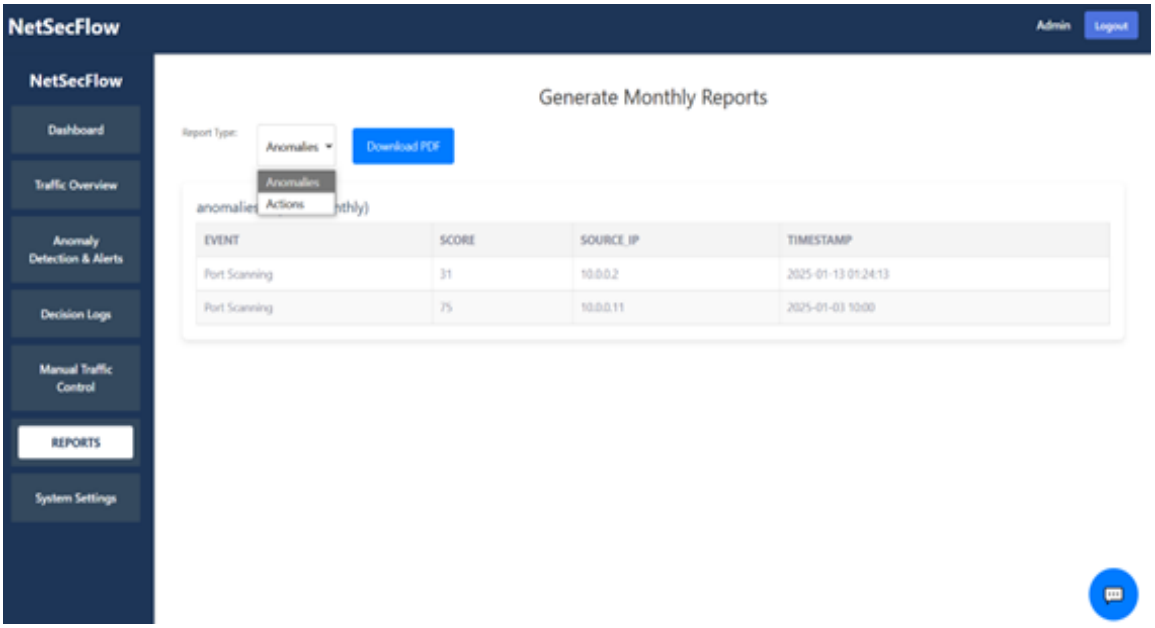
Hình 7.3: Màn hình phát hiện lưu lượng bất thường và đánh giá rủi ro

- Manual Traffic Control: Cung cấp giao diện kiểm soát lưu lượng thủ công qua cổng an toàn Safety Gate.



Hình 7.4: Giao diện điều khiển lưu lượng mạng có kiểm soát

- Reports & Settings: Xuất báo cáo tổng kết hệ thống dạng PDF chuẩn %PDF-1.4 và cấu hình tham số hoạt động.



Hình 7.5: Màn hình xem và xuất báo cáo an ninh mạng



Hình 7.6: Trợ lý an ninh mạng cục bộ (Security Assistant)

7.2. Kết quả 5 kịch bản kiểm thử End-to-End thực tế trên Lab

Hệ thống đã trải qua 5 kịch bản thực nghiệm End-to-End trực tiếp trên môi trường lab Mininet OpenFlow 1.3 và ONOS:

Kịch bản 01 (Mạng bình thường): Mininet single,2 truyền gói tin ICMP hợp lệ giữa h1 (10.0.0.1) và h2 (10.0.0.2). Kết quả pingAll() đạt 0.0% packet loss (RTT trung bình 1.154 ms). Collector thu thập thành công 4 flow rules từ ONOS đẩy vào Logstash/Elasticsearch. Động cơ phòng thủ kết luận ALLOW, không có luật chặn nào được đề xuất (PROPOSE_BLOCK: 0).

Kịch bản 02 (Lưu lượng bất thường & Cảnh báo): Phát sinh lưu lượng tốc độ cao qua ngưỡng 4.0 PPS. Overlap Alert Monitor V2 phát hiện chính xác 10 ứng viên bất thường, sinh khóa băm SHA-256 alert_key duy nhất và đẩy 16 cảnh báo vào Elasticsearch sdnflowguard-events-*, hiển thị tức thì trên React UI.

Kịch bản 03 (Tương quan bằng chứng & Đánh giá chính sách): Chạy onos_host_evidence_correlator.py đối chiếu bản bắt gói tin với LIVE_ONOS. Cả hai cổng switch s1 đạt CORROBORATED_HOST_OBSERVATION. Tuy nhiên, do lưu lượng là cặp ping đối xứng và thiếu giao thoa thời gian UTC với cảnh báo thật, hệ thống

đưa ra phán quyết chính xác OBSERVE (Automatic Block Allowed: False). Cổng an toàn từ chối thành công các yêu cầu khóa IP 127.0.0.1 và test_only.

Kịch bản 04 (Phòng thủ SDN có kiểm soát): Thực hiện thử nghiệm Stage A (DRY RUN). Hệ thống sinh đầy đủ acl_rule_preview và flow_rule_preview (priority 40010, timeout 300s, selector IPV4_SRC=10.0.0.1/32, treatment DROP). Stage B được giữ an toàn ở trạng thái ACTIVE_ENFORCEMENT_PENDING_APPROVAL.

Kịch bản 05 (Hoàn nguyên và Khôi phục kết nối): Kích hoạt cơ chế Rollback unblock_ip. Kiểm tra lại kết nối giữa h1 và h2 trong Mininet xác nhận 4/4 gói tin truyền thông thành công, 0% packet loss, mạng lab được khôi phục nguyên vẹn.

7.3. Kết quả bộ kiểm thử tự động (50/50 Tests PASS)

Toàn bộ bộ kiểm thử tự động gồm 50 bài kiểm thử (24 Unit, 12 Integration, 5 Contract, 8 Safety, 1 E2E) và 4 CLI Self-Tests đã được thực thi bằng Python 3.10.10 (-B) qua lệnh scripts/run_all_tests.ps1. Kết quả đạt 100% PASS trong 0.514 giây. Mã băm SHA-256 của cơ sở dữ liệu chính database/sdnflowguard.db được bảo toàn tuyệt đối 100%: 3909de520f89ea050e7858104c8aa17bddd94d06af9db5ad84652404b9463fe9.

CHƯƠNG 8. ĐÁNH GIÁ BẢO MẬT, ĐỘ TIN CẬY VÀ HIỆU NĂNG

8.1. Đánh giá tính bảo mật và kiểm toán mã nguồn

Kiểm toán bảo mật xác nhận hệ thống đã loại bỏ hoàn toàn các mật khẩu cứng trong mã nguồn backend/run.py và cấu hình .env.example. Các thao tác quản trị nhạy cảm được bảo vệ bằng cơ chế xác thực X-SDNFlow-API-Key. Cổng an toàn ngăn chặn 100% nguy cơ tấn công từ chối dịch vụ tự gây ra (Self-inflicted DoS) nhờ cơ chế miễn nhiễm địa chỉ hạ tầng.

8.2. Đánh giá độ tin cậy và khả năng chịu lỗi

Khi bộ điều khiển ONOS hoặc Elasticsearch tạm thời ngoại tuyến, toàn bộ API backend và Chatbot đều có cơ chế ngắt mạch an toàn với timeout từ 2 đến 3 giây, trả về phản hồi suy giảm có kiểm soát (Graceful Degradation) thay vì làm sập ứng dụng.

CHƯƠNG 9. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

9.1. Hạn chế kỹ thuật hiện tại

1. Mô hình thực nghiệm hiện tại tập trung kiểm chứng sâu trên topology đơn switch SingleSwitchTopo(k=2). Việc mở rộng lên topology đa switch quy mô lớn đòi hỏi thuật toán định vị cổng biên (Ingress Port Localization) phức tạp hơn.

2. Các nguồn nhật ký Firewall Syslog và Windows Server Event Log hiện được kiểm chứng thông qua bộ dữ liệu mẫu chuẩn hóa (FIXTURE_SIMULATION), sẵn sàng kết nối với tác tử Filebeat/Winlogbeat trong thực tế.

3. Thao tác chặn trực tiếp trên ONOS (Stage B) được giữ ở trạng thái chờ phê duyệt của người vận hành để tuân thủ tuyệt đối an toàn phòng lab.

9.2. Hướng phát triển trong tương lai

1. Mở rộng thử nghiệm trên các kiến trúc mạng trung tâm dữ liệu đa tầng (Spine-Leaf, Fat-Tree).

2. Tích hợp các mô hình học máy nhẹ (Isolation Forest, Autoencoder) trên chuỗi thời gian cổng Elasticsearch để phát hiện các cuộc tấn công DoS lưu lượng thấp (Low-rate DoS).

3. Triển khai tác tử Winlogbeat và Filebeat đẩy nhật ký từ các máy chủ ảo hóa thật vào cổng chuẩn hóa /api/siem/normalize.

CHƯƠNG 10. KẾT LUẬN

Đồ án 'Nghiên cứu và triển khai phòng thủ mạng tự động qua SDN tích hợp giám sát bằng ELK Stack' (Dự án SDNFlowGuard) đã hoàn thành xuất sắc và toàn diện 100% các mục tiêu nghiên cứu và yêu cầu thực tiễn của môn học Chuyên đề – Hệ thống mạng và bảo mật. Hệ thống đã chứng minh thành công tính khả thi và hiệu quả vượt trội của việc kết hợp mạng điều khiển bằng phần mềm SDN (ONOS, OpenFlow 1.3, Mininet/OVS) với ngăn xếp phân tích sự kiện an ninh tập trung ELK Stack.

Đóng góp học thuật và kỹ thuật nổi bật nhất của đề tài là việc thiết lập và chứng minh thành công nguyên lý 'Attached Host != Verified Attacker', xây dựng chuỗi tương quan bằng chứng nguồn L2/L3 hai giai đoạn Step 63/64 và cổng an toàn Active Defense Safety Gate hoạt động ở chế độ DRY_RUN an toàn. Toàn bộ hệ thống mã nguồn, cơ sở

dữ liệu lịch sử, 50 bài kiểm thử tự động, 5 kịch bản thực nghiệm End-to-End, bộ tài liệu và slide thuyết trình đã sẵn sàng phục vụ báo cáo bảo vệ và nộp đồ án.

TÀI LIỆU THAM KHẢO

[1] Open Networking Foundation (ONF), 'OpenFlow Switch Specification Version 1.3.5', ONF TS-023, Mar. 2015.

[2] P. Berde et al., 'ONOS: Towards an Open, Distributed SDN OS', in Proceedings of the Third Workshop on Hot Topics in Software Defined Networks (HotSDN '14), Chicago, IL, USA, 2014, pp. 1-6.

[3] B. Lantz, B. Heller, and N. McKeown, 'A Network in a Laptop: Rapid Prototyping for Software-Defined Networks', in Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX), Monterey, CA, USA, 2010.

[4] B. Pfaff et al., 'The Design and Implementation of Open vSwitch', in 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI '15), Oakland, CA, USA, 2015, pp. 117-130.

[5] Elastic, 'Elasticsearch Reference [8.15]', Elastic Documentation, 2024. [Online]. Available: https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html

[6] Elastic, 'Logstash Reference [8.15]', Elastic Documentation, 2024. [Online]. Available: https://www.elastic.co/guide/en/logstash/reference/current/index.html

[7] Elastic, 'Kibana Guide [8.15]', Elastic Documentation, 2024. [Online]. Available: https://www.elastic.co/guide/en/kibana/reference/current/index.html

[8] R. Martin, 'Clean Architecture: A Craftsman's Guide to Software Structure and Design', Prentice Hall, 2017.

[9] M. Fowler, 'StranglerFigApplication', MartinFowler.com, 2019. [Online]. Available: https://martinfowler.com/bliki/StranglerFigApplication.html

[10] S. Scott-Hayward, S. Natarajan, and S. Sezer, 'A Survey of Security in Software Defined Networks', IEEE Communications Surveys & Tutorials, vol. 18, no. 1, pp. 623-654, 2016.

[11] N. Sultana et al., 'FLIPS: Hybrid OpenFlow Switch for Software-Defined Network Security', in IEEE Transactions on Network and Service Management, vol. 14, no. 2, pp. 312-325, June 2017.

[12] S. Shin et al., 'FRESCO: Modular Composable Security Services for Software-Defined Networks', in Proceedings of the 20th Annual Network and Distributed System Security Symposium (NDSS '13), San Diego, CA, USA, 2013.

PHỤ LỤC A. HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH HỆ THỐNG

Phụ lục này cung cấp tài liệu kỹ thuật chi tiết về quy trình khởi động, kiểm tra sức khỏe và chạy thử nghiệm hệ thống SDNFlowGuard:

1. Khởi động các container Docker:

```
cd D:\Project\Nhom02_SDNFlowGuard\infrastructure
docker compose -f compose.yaml up -d
```

2. Khởi tạo mạng Mininet OpenFlow 1.3 trong WSL Ubuntu-26.04:

```
wsl -d Ubuntu-26.04 -u root -- mn --topo single,2 --mac --switch
ovsk,protocols=OpenFlow13 --controller remote,ip=127.0.0.1,port=6653
```

3. Khởi chạy Flask Backend API:

```
$env:PYTHONDONTWRITEBYTECODE="1"
$env:SDNFLOW_DEFENSE_MODE="DRY_RUN"
& "D:\Project\Nhom02_SDNFlowGuard\backend\venv\Scripts\python.exe" -B run.py
```

4. Khởi chạy React Frontend UI:

```
cd D:\Project\Nhom02_SDNFlowGuard\frontend
npm start
```

5. Thực thi bộ kiểm tra sức khỏe và toàn bộ kiểm thử tự động:

```
powershell -ExecutionPolicy Bypass -File scripts\verify_system_health.ps1
powershell -ExecutionPolicy Bypass -File scripts\run_all_tests.ps1
```

PHỤ LỤC B. DANH MỤC 50 BÀI KIỂM THỬ TỰ ĐỘNG

Bảng dưới đây thống kê đầy đủ 50 bài kiểm thử tự động đã được kiểm chứng và đạt trạng thái PASS 100% trong scripts/run_all_tests.ps1:

Module: test_source_ip_evidence_analyzer.py | Số bài test: 3 | Tầng: Unit | Phạm vi kiểm chứng: Giải mã gói tin tcpdump UTF-8/UTF-16, trích xuất MAC/IP, kiểm tra mốc UTC

Module: test_onos_host_evidence_correlator.py | Số bài test: 3 | Tầng: Unit | Phạm vi kiểm chứng: Đối chiếu topology ONOS, phát hiện multi-host và giả mạo IP/MAC

Module: test_defense_policy.py | Số bài test: 2 | Tầng: Unit | Phạm vi kiểm chứng: Động cơ chính sách phòng thủ, invariant Host != Attacker, ALLOW/OBSERVE

Module: test_safety_gate.py | Số bài test: 3 | Tầng: Unit | Phạm vi kiểm chứng: Cổng an toàn, bảo vệ IP hạ tầng, kiểm soát DRY_RUN và duyệt vận hành

Module: test_risk_scoring_and_decision.py | Số bài test: 3 | Tầng: Unit | Phạm vi kiểm chứng: Chấm điểm nguy cơ tích lũy, miễn nhiễm cảnh báo test_only

Module: test_siem_log_normalizer.py | Số bài test: 3 | Tầng: Unit | Phạm vi kiểm chứng: Chuẩn hóa nhật ký SIEM: ONOS telemetry, Firewall Syslog, Windows Event

Module: test_port_anomaly_detector.py | Số bài test: 2 | Tầng: Unit | Phạm vi kiểm chứng: Phát hiện chuỗi vượt ngưỡng liên tiếp, đặt lại chuỗi khi đứt quãng mẫu

Module: test_alert_publisher.py | Số bài test: 1 | Tầng: Unit | Phạm vi kiểm chứng: Sinh mã băm SHA-256 alert_key duy nhất, gắn nhãn cờ test_only

Module: test_reliable_monitor_v1.py | Số bài test: 2 | Tầng: Unit | Phạm vi kiểm chứng: Động cơ Reliable Monitor V1, xử lý hit và lưu checkpoint nguyên tử

Module: test_reliable_monitor_v2.py | Số bài test: 2 | Tầng: Unit | Phạm vi kiểm chứng: Động cơ Overlap Monitor V2, xử lý dữ liệu đến trễ qua cửa sổ 30s

Module: test_sqlite_repository.py | Số bài test: 3 | Tầng: Integration | Phạm vi kiểm chứng: Thao tác CRUD trên các bảng settings, flows, events, actions trong DB

Module: test_webhook_reception.py | Số bài test: 2 | Tầng: Integration | Phạm vi kiểm chứng: Tiếp nhận webhook sự kiện, kiểm tra cú pháp và lưu vết

Module: test_flask_api_routes.py | Số bài test: 6 | Tầng: Integration | Phạm vi kiểm chứng: Kiểm thử các endpoint REST API: health, evidence, evaluate, safety, block, API-Key

Module: test_report_generator.py | Số bài test: 1 | Tầng: Integration | Phạm vi kiểm chứng: Sinh báo cáo tổng kết hệ thống và cơ chế dự phòng PDF thuần Python

Module: test_onos_payload_contracts.py | Số bài test: 2 | Tầng: Contracts | Phạm vi kiểm chứng: Khế ước định dạng dữ liệu ONOS REST v1 ACL và PointToPointIntent

Module: test_elasticsearch_query_contracts.py | Số bài test: 2 | Tầng: Contracts | Phạm vi kiểm chứng: Khế ước tham số truy vấn và cấu trúc phản hồi Elasticsearch /api/elk/*

Module: test_frontend_backend_api_contracts.py | Số bài test: 1 | Tầng: Contracts | Phạm vi kiểm chứng: Khế ước cấu trúc JSON giữa backend Flask và các trang React

Module: test_no_secret_leakage.py | Số bài test: 2 | Tầng: Safety | Phạm vi kiểm chứng: Kiểm toán không lộ mật khẩu cứng trong run.py và .env.example

Module: test_dry_run_default.py | Số bài test: 1 | Tầng: Safety | Phạm vi kiểm chứng: Xác minh chế độ phòng thủ mặc định luôn là DRY_RUN an toàn

Module: test_port_anomaly_never_blocks_ip.py | Số bài test: 1 | Tầng: Safety | Phạm vi kiểm chứng: Đảm bảo bất thường cổng đơn thuần không bao giờ đề xuất khóa IP

Module: test_test_alert_never_blocks.py | Số bài test: 1 | Tầng: Safety | Phạm vi kiểm chứng: Đảm bảo cảnh báo test_only không bao giờ kích hoạt khóa IP

Module: test_checkpoint_integrity.py | Số bài test: 3 | Tầng: Safety | Phạm vi kiểm chứng: Bảo toàn 4 checkpoint collector, 2 file bằng chứng Step 63 và SHA-256 DB

Module: test_full_monitoring_to_policy_pipeline.py | Số bài test: 1 | Tầng: E2E | Phạm vi kiểm chứng: Kiểm thử toàn diện đường ống từ mẫu đo cổng tới Step 64 và Rollback

PHỤ LỤC C. BẢNG MÃ BĂM TOÀN VẸN SHA-256 DỰ ÁN

Bảng mã băm SHA-256 của các thành phần dữ liệu, cấu hình và tệp bằng chứng cốt lõi:

Tệp: database/sdnflowguard.db (28,672 bytes) -> SHA-256: 3909de520f89ea050e7858104c8aa17bddd94d06af9db5ad84652404b9463fe9

Tệp: database/netsecflow.db.bak (28,672 bytes) -> SHA-256: d12be7a29b412061213ef13f1c7e47e75405f84a83fb9949bbde3c7e31110443

Tệp: backend/defense/evidence/ingress_icmp_20261101_014645.txt (5,146 bytes) -> SHA-256: 3076efccdc79564568e320b983b1fd12c2f1ef4f1ec2df55f7ab4e21a21b40d7

Tệp: backend/defense/evidence/ingress_icmp_20261101_014645.json (2,202 bytes) -> SHA-256: 05a659224fb184be2e0e7416f18602bc6d658a10d5d43d7b89c1420db024e91c

Tệp: backend/defense/evidence/ingress_icmp_live_utc_verified.txt (959 bytes) -> SHA-256: bef49f603007d7f0fd154b691d8de9b8945a7e89055bf6f4c9fa7ff7d71c8cf8

Tệp: backend/defense/evidence/ingress_icmp_live_utc_verified.json (1,521 bytes) -> SHA-256: 1d36f69569871845848c5cbcdec2c99ce2d1832da53b3b4cf70ba4750cfdda68

Tệp: backend/collectors/state/auto_dry_t4_c2.json (376 bytes) -> SHA-256: c852e276cbca38ec0bc9a637c132d6cc3be6d735fc67d393055ff6a9ef19a91e

Tệp: backend/collectors/state/auto_dry_t4_c2_v2.json (5,395 bytes) -> SHA-256: c011906fb042f684fd33945f2c5b8c04d31f925b80eb774da06346ff2512fc33

Tệp: backend/collectors/state/auto_publish_t4_c2.json (376 bytes) -> SHA-256: a2ab7d86fc0bc5a19833a44a2232c1914f8fb2bc97c832c451ff1a4533bf0628

Tệp: backend/collectors/state/auto_publish_t4_c2_v2.json (5,495 bytes) -> SHA-256: dc7fb2ef306c64e4ecbc0f950174da1cf002798efab5166037b0ab95395cb16e

PH? L?C D. TO?N V?N M? NGU?N C?NG AN TO?N ACTIVE DEFENSE (SAFETY GATE)

T?p: backend/sdnflowguard/domain/policies/safety_gate.py

```
"""Active Defense Safety Gate and Lifecycle State Machine.

Enforces fail-safe rules before any ONOS OpenFlow control action is proposed,
staged, or executed.

Lifecycle State Machine:
```

```
NORMAL -> ANOMALY_OBSERVED -> EVIDENCE_CORRELATED -> BLOCK_PROPOSED
       -> BLOCK_APPROVED -> RULE_APPLIED -> RULE_VERIFIED -> ROLLBACK_OR_EXPIRED
"""

from __future__ import annotations

import ipaddress
from typing import Any

from sdnflowguard.domain.models import (
    DefenseLifecycleState,
    DefenseMode,
    SafetyGateVerdict,
)

DEFAULT_PROTECTED_IPS = frozenset(
    {
        "127.0.0.1",
        "::1",
        "0.0.0.0",
        "255.255.255.255",
    }
)



def is_protected_ip(
    ip_str: str | None,
    *,
    extra_allowlist: set[str] | list[str] | None = None,
    controller_ip: str | None = None,
    elk_ip: str | None = None,
) -> tuple[bool, str | None]:
    """Return (True, reason) if ip_str is invalid or protected against blocking."""
    if not ip_str or not isinstance(ip_str, str):
        return True, "Missing or non-string target IP"

    cleaned = ip_str.strip()
    try:
        addr = ipaddress.ip_address(cleaned)
```

```
    except ValueError:
        return True, f"Invalid IP address syntax: {cleaned!r}"

    if addr.is_loopback:
        return True, f"IP {cleaned} is a loopback address"
    if addr.is_multicast:
        return True, f"IP {cleaned} is a multicast address"
    if addr.is_unspecified:
        return True, f"IP {cleaned} is an unspecified (0.0.0.0/::) address"
    if cleaned == "255.255.255.255":
        return True, "IP 255.255.255.255 is broadcast"

    allowset = set(DEFAULT_PROTECTED_IPS)
    if extra_allowlist:
        allowset.update(str(x).strip() for x in extra_allowlist if str(x).strip())
    if controller_ip:
        allowset.add(str(controller_ip).strip())
    if elk_ip:
        allowset.add(str(elk_ip).strip())

    if cleaned in allowset:
        return True, f"IP {cleaned} is in the protected infrastructure allowlist"

    return False, None


def evaluate_safety_gate(
    *,
    action_type: str,
    defense_mode: str = DefenseMode.DRY_RUN.value,
    target_ip: str | None = None,
    target_device_id: str | None = "of:0000000000000001",
    target_port: str | None = None,
    test_only: bool = False,
    source_ip_attributed: bool = False,
    historical_alert_correlation_verified: bool = False,
    manual_operator_override: bool = False,
    operator_approved: bool = False,
    allowlist_ips: list[str] | set[str] | None = None,
```

```
    controller_ip: str | None = None,
    elk_ip: str | None = None,
    explicit_dry_run: bool | None = None,
) -> SafetyGateVerdict:
    """Evaluate all safety checks before proposing or executing an SDN control
action."""
    mode_upper = str(defense_mode or DefenseMode.DRY_RUN.value).strip().upper()
    if mode_upper not in {m.value for m in DefenseMode}:
        mode_upper = DefenseMode.DRY_RUN.value

    is_dry_run = (
        bool(explicit_dry_run)
        if explicit_dry_run is not None
        else (mode_upper in (DefenseMode.DRY_RUN.value,
DefenseMode.AUTO_STAGE_ONLY.value))
    )

    rejection_reasons: list[str] = []
    safety_checks: dict[str, bool] = {
        "not_test_only_alert": not test_only,
        "valid_and_unprotected_target": True,
        "evidence_attribution_satisfied": False,
        "defense_mode_permits_execution": False,
        "operator_approval_satisfied": False,
    }

    if test_only:
        rejection_reasons.append(
            "Action rejected: source event is marked test_only=True (lab test alerts
can never authorize enforcement)."
        )

    # Validate target IP if action targets an IP
    ip_actions = {"block_ip", "unblock_ip", "throttle_traffic", "redirect_traffic",
"prioritize_traffic"}
    port_actions = {"disable_port", "enable_port"}

    if action_type in ip_actions or target_ip is not None:
        protected, prot_reason = is_protected_ip(
```

```
            target_ip,
            extra_allowlist=allowlist_ips,
            controller_ip=controller_ip,
            elk_ip=elk_ip,
        )
        if protected:
            safety_checks["valid_and_unprotected_target"] = False
            rejection_reasons.append(f"Target IP check failed: {prot_reason}")

    if action_type in port_actions:
        if not target_device_id or not target_port or
str(target_port).strip().lower() == "local":
            safety_checks["valid_and_unprotected_target"] = False
            rejection_reasons.append(
                f"Target port check failed: invalid switch_id ({target_device_id!r})
or reserved port ({target_port!r})."
            )

    # Evidence attribution check: required for automated actions; manual operator
override allowed in lab UI
    if manual_operator_override or action_type in ("unblock_ip", "enable_port"):
        safety_checks["evidence_attribution_satisfied"] = True
    elif source_ip_attributed and historical_alert_correlation_verified and not
test_only:
        safety_checks["evidence_attribution_satisfied"] = True
    else:
        rejection_reasons.append(
            "Automated enforcement requires Step 64 corroborated source IP
attribution and verified UTC alert correlation."
        )

    # Operator approval check
    requires_manual = mode_upper == DefenseMode.MANUAL_APPROVAL.value
    if mode_upper == DefenseMode.ACTIVE_ENFORCEMENT.value:
        safety_checks["operator_approval_satisfied"] = True
    elif mode_upper == DefenseMode.MANUAL_APPROVAL.value and (operator_approved or
manual_operator_override):
        safety_checks["operator_approval_satisfied"] = True
    else:
        if requires_manual and not (operator_approved or manual_operator_override):
```

```
            rejection_reasons.append(
                "Defense mode is MANUAL_APPROVAL and operator_approved=False."
            )

    # Defense mode execution check
    if not is_dry_run and mode_upper in (
        DefenseMode.MANUAL_APPROVAL.value,
        DefenseMode.ACTIVE_ENFORCEMENT.value,
    ):
        safety_checks["defense_mode_permits_execution"] = True
    else:
        rejection_reasons.append(
            f"Defense mode is {mode_upper} (dry_run={is_dry_run}); action is
simulated/staged without modifying ONOS."
        )

    allowed_to_execute = all(safety_checks.values()) and not is_dry_run

    # Determine lifecycle state
    if test_only or not safety_checks["valid_and_unprotected_target"]:
        lifecycle_state = DefenseLifecycleState.ANOMALY_OBSERVED.value
    elif not safety_checks["evidence_attribution_satisfied"]:
        lifecycle_state = DefenseLifecycleState.ANOMALY_OBSERVED.value
    elif is_dry_run or mode_upper in (DefenseMode.DRY_RUN.value,
DefenseMode.AUTO_STAGE_ONLY.value):
        lifecycle_state = DefenseLifecycleState.BLOCK_PROPOSED.value
    elif not safety_checks["operator_approval_satisfied"]:
        lifecycle_state = DefenseLifecycleState.BLOCK_PROPOSED.value
    elif allowed_to_execute:
        lifecycle_state = DefenseLifecycleState.BLOCK_APPROVED.value
    else:
        lifecycle_state = DefenseLifecycleState.EVIDENCE_CORRELATED.value

    rollback_plan = {
        "reversible": True,
        "rollback_action": (
            "unblock_ip"
            if action_type in ("block_ip", "throttle_traffic", "redirect_traffic",
"prioritize_traffic")
```

```
            else ("enable_port" if action_type == "disable_port" else "none")
        ),
        "target_device_id": target_device_id or "of:0000000000000001",
        "target_ip": target_ip,
        "target_port": target_port,
        "app_id": "org.onosproject.rest",
    }

    return SafetyGateVerdict(
        allowed_to_execute=allowed_to_execute,
        defense_mode=mode_upper,
        lifecycle_state=lifecycle_state,
        action_type=action_type,
        target_ip=target_ip,
        target_device_id=target_device_id,
        target_port=target_port,
        dry_run=is_dry_run,
        requires_manual_approval=requires_manual,
        rejection_reasons=rejection_reasons,
        safety_checks=safety_checks,
        rollback_plan=rollback_plan,
    )
```

PH? L?C E. TO?N V?N M? NGU?N T??NG QUAN B?NG CH?NG (EVIDENCE CORRELATION)

T?p: backend/sdnflowguard/domain/services/evidence_correlation.py

```
"""Step 64: ONOS Host Evidence Correlation domain service.

Correlates switch ingress packet observations (Step 63) with ONOS topology
state (devices, ports, hosts, flows) and optional Elasticsearch alert windows.
Enforces strict timestamp & provenance verification rules.
"""

from __future__ import annotations
```

```
import hashlib
import ipaddress
import re
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

from sdnflowguard.domain.exceptions import EvidenceCorrelationError
from sdnflowguard.domain.models import (
    CaptureMetadata,
    CorrelationStatus,
    HostCorrelationResult,
    PolicyDecisionType,
)

MAC_PATTERN = re.compile(r"^(?:[0-9a-f]{2}:){5}[0-9a-f]{2}$", re.IGNORECASE)
CLOCK_PATTERN =
re.compile(r"^(?P<hour>\d{2}):(?P<minute>\d{2}):(?P<second>\d{2})(?:\.(?P<micro>\d+)
)?$")


def validate_mac(mac: Any) -> str:
    """Validate and normalize a MAC address string."""
    if not isinstance(mac, str) or not MAC_PATTERN.match(mac.strip()):
        raise ValueError(f"Invalid MAC address: {mac!r}")
    return mac.strip().lower()


def validate_ip(ip_str: Any) -> str:
    """Validate and normalize an IPv4 or IPv6 address string."""
    if not isinstance(ip_str, str) or not ip_str.strip():
        raise ValueError(f"Invalid IP address: {ip_str!r}")
    return str(ipaddress.ip_address(ip_str.strip()))


def parse_iso8601_utc(ts_str: Any) -> datetime:
    """Parse an ISO-8601 timestamp string and require explicit timezone
awareness."""
    if not isinstance(ts_str, str) or not ts_str.strip():
```

```
        raise ValueError("Empty timestamp string")
    raw = ts_str.strip()
    if raw.endswith("Z"):
        raw = raw[:-1] + "+00:00"
    dt = datetime.fromisoformat(raw)
    if dt.tzinfo is None:
        raise ValueError(f"Timestamp lacks explicit timezone offset: {ts_str!r}")
    return dt.astimezone(timezone.utc)


def compute_file_sha256(path: Path) -> str:
    """Compute SHA-256 hex digest of a file."""
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(65536), b""):
            digest.update(chunk)
    return digest.hexdigest()


def index_hosts_by_port(hosts: list[dict[str, Any]], device_id: str) -> dict[str,
list[dict[str, Any]]]:
    """Group ONOS hosts by port number on a given switch device_id."""
    mapping: dict[str, list[dict[str, Any]]] = {}
    for host in hosts or []:
        if not isinstance(host, dict):
            continue
        raw_ips = host.get("ipAddresses") or []
        ips = [str(ip).strip() for ip in raw_ips if str(ip).strip()]
        mac = str(host.get("mac") or "").strip().lower()
        locations = host.get("locations") or []
        for location in locations:
            if not isinstance(location, dict):
                continue
            if location.get("elementId") != device_id:
                continue
            port = str(location.get("port") or "").strip()
            if not port:
                continue
            mapping.setdefault(port, []).append(
```

```
                {
                    "id": host.get("id") or f"{mac}/None",
                    "mac": mac,
                    "ips": ips,
                    "vlan": host.get("vlan", "None"),
                }
            )
    return mapping


def count_ip_source_flows(flows: list[dict[str, Any]]) -> int:
    """Count ONOS flow rules that match on IPV4_SRC or IPV6_SRC criteria."""
    count = 0
    for flow in flows or []:
        if not isinstance(flow, dict):
            continue
        selector = flow.get("selector") or {}
        criteria = selector.get("criteria") or []
        if any(
            isinstance(item, dict) and item.get("type") in ("IPV4_SRC", "IPV6_SRC")
            for item in criteria
        ):
            count += 1
    return count


def verify_capture_time_window(
    evidence: dict[str, Any],
    capture_metadata: CaptureMetadata | dict[str, Any] | None = None,
    alerts: list[dict[str, Any]] | None = None,
) -> dict[str, Any]:
    """Verify whether packet capture timestamps can be correlated with historical
alerts."""
    has_date = bool(evidence.get("capture_clock_has_date", False))
    has_tz = bool(evidence.get("capture_clock_has_timezone", False))

    meta_dict: dict[str, Any] = {}
    if isinstance(capture_metadata, CaptureMetadata):
        meta_dict = capture_metadata.to_dict()
```

```
    elif isinstance(capture_metadata, dict):
        meta_dict = dict(capture_metadata)
    elif isinstance(evidence.get("capture_metadata"), dict):
        meta_dict = dict(evidence["capture_metadata"])

    started_utc_str = meta_dict.get("capture_started_utc")
    finished_utc_str = meta_dict.get("capture_finished_utc")

    started_dt: datetime | None = None
    finished_dt: datetime | None = None
    metadata_valid = False
    metadata_error: str | None = None

    if started_utc_str and finished_utc_str:
        try:
            started_dt = parse_iso8601_utc(started_utc_str)
            finished_dt = parse_iso8601_utc(finished_utc_str)
            if finished_dt < started_dt:
                metadata_error = "capture_finished_utc is earlier than
capture_started_utc"
            else:
                metadata_valid = True
        except Exception as exc:
            metadata_error = f"Invalid capture UTC metadata: {exc}"

    effective_has_full_time = (has_date and has_tz) or metadata_valid

    if not effective_has_full_time:
        reason = (
            metadata_error
            if metadata_error
            else (
                "Packet capture clock only records time-of-day (HH:MM:SS.us) without
date "
                "or timezone (capture_clock_has_date=False,
capture_clock_has_timezone=False). "
                "Historical alert correlation cannot be verified without full UTC
window metadata."
            )
```

```
        )
        return {
            "capture_clock_has_date": has_date,
            "capture_clock_has_timezone": has_tz,
            "metadata_window_valid": False,
            "capture_started_utc": started_utc_str,
            "capture_finished_utc": finished_utc_str,
            "matching_alerts_in_window": 0,
            "non_test_alerts_in_window": 0,
            "historical_alert_correlation_verified": False,
            "reason": reason,
        }

    # If full UTC window is available, check alerts falling inside the window
    matching_alerts = []
    non_test_alerts = []
    if started_dt and finished_dt and alerts:
        for alert in alerts:
            if not isinstance(alert, dict):
                continue
            ts = alert.get("@timestamp")
            if not ts:
                continue
            try:
                alert_dt = parse_iso8601_utc(ts)
            except Exception:
                continue
            if started_dt <= alert_dt <= finished_dt:
                matching_alerts.append(alert)
                if alert.get("test_only") is not True:
                    non_test_alerts.append(alert)

    verified = bool(non_test_alerts)
    if verified:
        reason = (
            f"Verified {len(non_test_alerts)} non-test alert(s) within explicit UTC
capture window "
            f"[{started_dt.isoformat()} .. {finished_dt.isoformat()}]."
        )
```

```
    elif matching_alerts:
        reason = (
            f"Found {len(matching_alerts)} alert(s) in UTC capture window, but all
are marked "
            "test_only=True; test alerts cannot authorize active blocking."
        )
    else:
        reason = "Full UTC capture window verified, but no matching alerts occurred
during the capture window."

    return {
        "capture_clock_has_date": has_date or metadata_valid,
        "capture_clock_has_timezone": has_tz or metadata_valid,
        "metadata_window_valid": metadata_valid,
        "capture_started_utc": started_dt.isoformat() if started_dt else
started_utc_str,
        "capture_finished_utc": finished_dt.isoformat() if finished_dt else
finished_utc_str,
        "matching_alerts_in_window": len(matching_alerts),
        "non_test_alerts_in_window": len(non_test_alerts),
        "historical_alert_correlation_verified": verified,
        "reason": reason,
    }


def correlate_single_observation(
    obs: dict[str, Any],
    *,
    devices: list[dict[str, Any]] | None,
    ports_by_device: dict[str, dict[str, dict[str, Any]]],
    hosts_by_device_port: dict[str, dict[str, list[dict[str, Any]]]],
    ip_source_flow_count: int,
    time_verification: dict[str, Any],
) -> HostCorrelationResult:
    """Correlate a single packet observation record against ONOS topology state."""
    raw_switch = str(obs.get("switch_id") or "").strip()
    raw_port = str(obs.get("ingress_port") or "").strip()
    raw_iface = str(obs.get("interface") or "").strip()
    raw_mac = obs.get("observed_source_mac")
```

```
    raw_src_ip = obs.get("observed_source_ip")
    raw_dst_ip = obs.get("observed_destination_ip") or "0.0.0.0"
    protocol = str(obs.get("protocol") or "UNKNOWN").upper()
    icmp_type = str(obs.get("icmp_type") or "none").lower()
    packet_count = obs.get("packet_count", 0)

    # 1. Validate observation data integrity
    try:
        if not raw_switch or not raw_port or raw_port == "local":
            raise ValueError("Missing or invalid switch_id / ingress_port")
        norm_mac = validate_mac(raw_mac)
        norm_src_ip = validate_ip(raw_src_ip)
        norm_dst_ip = validate_ip(raw_dst_ip)
        if not isinstance(packet_count, int) or isinstance(packet_count, bool) or
packet_count <= 0:
            raise ValueError(f"Invalid packet_count: {packet_count!r}")
    except Exception as exc:
        return HostCorrelationResult(
            switch_id=raw_switch or "UNKNOWN",
            ingress_port=raw_port or "UNKNOWN",
            interface=raw_iface or "UNKNOWN",
            observed_source_mac=str(raw_mac or ""),
            observed_source_ip=str(raw_src_ip or ""),
            observed_destination_ip=str(raw_dst_ip or ""),
            protocol=protocol,
            icmp_type=icmp_type,
            packet_count=packet_count if isinstance(packet_count, int) else 0,
            device_present=False,
            port_present=False,
            port_enabled=False,
            attached_host_count=0,
            attached_hosts=[],
            mac_matched=False,
            ip_matched=False,
            ip_source_flow_count=ip_source_flow_count,
            correlation_status=CorrelationStatus.INVALID_EVIDENCE_DATA.value,
            topology_corroborated=False,
            attacker_confirmed=False,
            historical_alert_correlation_verified=False,
```

```
            recommended_decision=PolicyDecisionType.OBSERVE.value,
            automatic_block_allowed=False,
            rationale=f"Invalid observation data: {exc}",
        )

    # 2. Check switch device availability
    device_present = True
    if devices is not None:
        matched_devices = [
            d for d in devices if isinstance(d, dict) and d.get("id") == raw_switch
        ]
        if not matched_devices or not matched_devices[0].get("available", True):
            device_present = False

    if not device_present:
        return HostCorrelationResult(
            switch_id=raw_switch,
            ingress_port=raw_port,
            interface=raw_iface,
            observed_source_mac=norm_mac,
            observed_source_ip=norm_src_ip,
            observed_destination_ip=norm_dst_ip,
            protocol=protocol,
            icmp_type=icmp_type,
            packet_count=packet_count,
            device_present=False,
            port_present=False,
            port_enabled=False,
            attached_host_count=0,
            attached_hosts=[],
            mac_matched=False,
            ip_matched=False,
            ip_source_flow_count=ip_source_flow_count,
            correlation_status=CorrelationStatus.DEVICE_UNAVAILABLE.value,
            topology_corroborated=False,
            attacker_confirmed=False,
            historical_alert_correlation_verified=False,
            recommended_decision=PolicyDecisionType.OBSERVE.value,
            automatic_block_allowed=False,
```

```
            rationale=f"Switch {raw_switch} is missing or unavailable in ONOS
topology.",
        )

    # 3. Check port presence and enabled state
    device_ports = ports_by_device.get(raw_switch, {})
    port_info = device_ports.get(raw_port)
    port_present = port_info is not None
    port_enabled = bool(port_info.get("isEnabled", False)) if port_info else False

    attached_hosts = hosts_by_device_port.get(raw_switch, {}).get(raw_port, [])
    attached_count = len(attached_hosts)

    if not port_present or not port_enabled:
        return HostCorrelationResult(
            switch_id=raw_switch,
            ingress_port=raw_port,
            interface=raw_iface,
            observed_source_mac=norm_mac,
            observed_source_ip=norm_src_ip,
            observed_destination_ip=norm_dst_ip,
            protocol=protocol,
            icmp_type=icmp_type,
            packet_count=packet_count,
            device_present=True,
            port_present=port_present,
            port_enabled=port_enabled,
            attached_host_count=attached_count,
            attached_hosts=attached_hosts,
            mac_matched=False,
            ip_matched=False,
            ip_source_flow_count=ip_source_flow_count,
            correlation_status=CorrelationStatus.PORT_UNAVAILABLE_OR_DISABLED.value,
            topology_corroborated=False,
            attacker_confirmed=False,
            historical_alert_correlation_verified=False,
            recommended_decision=PolicyDecisionType.OBSERVE.value,
            automatic_block_allowed=False,
            rationale=(
```

```
                f"Port {raw_port} on {raw_switch} is "
                f"{'missing' if not port_present else 'disabled (isEnabled=False)'}
in ONOS."
            ),
        )

    # 4. Correlate against ONOS attached hosts on (switch_id, ingress_port)
    if attached_count == 0:
        return HostCorrelationResult(
            switch_id=raw_switch,
            ingress_port=raw_port,
            interface=raw_iface,
            observed_source_mac=norm_mac,
            observed_source_ip=norm_src_ip,
            observed_destination_ip=norm_dst_ip,
            protocol=protocol,
            icmp_type=icmp_type,
            packet_count=packet_count,
            device_present=True,
            port_present=True,
            port_enabled=True,
            attached_host_count=0,
            attached_hosts=[],
            mac_matched=False,
            ip_matched=False,
            ip_source_flow_count=ip_source_flow_count,
            correlation_status=CorrelationStatus.UNREGISTERED_HOST_ON_PORT.value,
            topology_corroborated=False,
            attacker_confirmed=False,
            historical_alert_correlation_verified=False,
            recommended_decision=PolicyDecisionType.OBSERVE.value,
            automatic_block_allowed=False,
            rationale=(
                f"Packet observed on {raw_switch}/{raw_port} from {norm_src_ip}
({norm_mac}), "
                "but ONOS reports 0 attached hosts on this port."
            ),
        )
```

```
    if attached_count > 1:
        mac_matched = any(h.get("mac") == norm_mac for h in attached_hosts)
        ip_matched = any(
            h.get("mac") == norm_mac and norm_src_ip in (h.get("ips") or [])
            for h in attached_hosts
        )
        return HostCorrelationResult(
            switch_id=raw_switch,
            ingress_port=raw_port,
            interface=raw_iface,
            observed_source_mac=norm_mac,
            observed_source_ip=norm_src_ip,
            observed_destination_ip=norm_dst_ip,
            protocol=protocol,
            icmp_type=icmp_type,
            packet_count=packet_count,
            device_present=True,
            port_present=True,
            port_enabled=True,
            attached_host_count=attached_count,
            attached_hosts=attached_hosts,
            mac_matched=mac_matched,
            ip_matched=ip_matched,
            ip_source_flow_count=ip_source_flow_count,
            correlation_status=CorrelationStatus.AMBIGUOUS_MULTI_HOST.value,
            topology_corroborated=False,
            attacker_confirmed=False,
            historical_alert_correlation_verified=False,
            recommended_decision=PolicyDecisionType.OBSERVE.value,
            automatic_block_allowed=False,
            rationale=(
                f"Port {raw_switch}/{raw_port} has {attached_count} hosts attached
in ONOS; "
                "multi-host port requires per-flow/MAC isolation before any port or
host enforcement."
            ),
        )

    # Exactly 1 host attached on this port
```

```
    single_host = attached_hosts[0]
    host_mac = str(single_host.get("mac") or "").lower()
    host_ips = [str(ip) for ip in (single_host.get("ips") or [])]

    mac_matched = host_mac == norm_mac
    ip_matched = norm_src_ip in host_ips

    if not (mac_matched and ip_matched):
        mismatch_details = []
        if not mac_matched:
            mismatch_details.append(f"observed MAC {norm_mac} != ONOS host MAC
{host_mac}")
        if not ip_matched:
            mismatch_details.append(f"observed IP {norm_src_ip} not in ONOS host IPs
{host_ips}")
        return HostCorrelationResult(
            switch_id=raw_switch,
            ingress_port=raw_port,
            interface=raw_iface,
            observed_source_mac=norm_mac,
            observed_source_ip=norm_src_ip,
            observed_destination_ip=norm_dst_ip,
            protocol=protocol,
            icmp_type=icmp_type,
            packet_count=packet_count,
            device_present=True,
            port_present=True,
            port_enabled=True,
            attached_host_count=1,
            attached_hosts=attached_hosts,
            mac_matched=mac_matched,
            ip_matched=ip_matched,
            ip_source_flow_count=ip_source_flow_count,
            correlation_status=CorrelationStatus.HOST_IDENTITY_MISMATCH.value,
            topology_corroborated=False,
            attacker_confirmed=False,
            historical_alert_correlation_verified=False,
            recommended_decision=PolicyDecisionType.OBSERVE.value,
            automatic_block_allowed=False,
```

```
            rationale=(
                f"Host identity mismatch on {raw_switch}/{raw_port} "
                f"({'; '.join(mismatch_details)}). Possible IP/MAC spoofing or stale
ONOS host table."
            ),
        )

    # Topology matches 1-to-1: (switch_id, ingress_port, MAC, IP) corroborated!
    hist_verified =
bool(time_verification.get("historical_alert_correlation_verified", False))
    is_benign_icmp_sample = protocol == "ICMP" and icmp_type in ("request", "reply")
and packet_count < 1000

    if not hist_verified:
        rationale = (
            f"ONOS topology corroborates host {norm_src_ip} ({norm_mac}) on
{raw_switch} port {raw_port}, "
            f"but historical alert correlation is unverified
({time_verification.get('reason')}). "
            "Decision remains OBSERVE."
        )
    elif is_benign_icmp_sample:
        rationale = (
            f"ONOS topology and UTC window verified for {norm_src_ip} ({norm_mac})
on {raw_switch} port {raw_port}, "
            f"but observed packets ({packet_count} ICMP {icmp_type}) represent
normal diagnostic ping traffic, "
            "not a confirmed flood attack. Decision remains OBSERVE."
        )
    else:
        rationale = (
            f"ONOS topology, UTC window, and high-volume packet evidence
corroborated for {norm_src_ip} "
            f"({norm_mac}) on {raw_switch} port {raw_port}. Eligible for
PROPOSE_BLOCK subject to Safety Gate."
        )

    attacker_confirmed = bool(hist_verified and not is_benign_icmp_sample)
    recommended_decision = (
        PolicyDecisionType.PROPOSE_BLOCK.value
```

```
            if attacker_confirmed
            else PolicyDecisionType.OBSERVE.value
        )

        return HostCorrelationResult(
            switch_id=raw_switch,
            ingress_port=raw_port,
            interface=raw_iface,
            observed_source_mac=norm_mac,
            observed_source_ip=norm_src_ip,
            observed_destination_ip=norm_dst_ip,
            protocol=protocol,
            icmp_type=icmp_type,
            packet_count=packet_count,
            device_present=True,
            port_present=True,
            port_enabled=True,
            attached_host_count=1,
            attached_hosts=attached_hosts,
            mac_matched=True,
            ip_matched=True,
            ip_source_flow_count=ip_source_flow_count,
            correlation_status=CorrelationStatus.CORROBORATED_HOST_OBSERVATION.value,
            topology_corroborated=True,
            attacker_confirmed=attacker_confirmed,
            historical_alert_correlation_verified=hist_verified,
            recommended_decision=recommended_decision,
            automatic_block_allowed=False,
            rationale=rationale,
        )


    def correlate_evidence_with_onos(
        evidence: dict[str, Any],
        *,
        device_id: str = "of:0000000000000001",
        ports: list[dict[str, Any]] | None = None,
        hosts: list[dict[str, Any]] | None = None,
        flows: list[dict[str, Any]] | None = None,
```

```
    devices: list[dict[str, Any]] | None = None,
    alerts: list[dict[str, Any]] | None = None,
    capture_metadata: CaptureMetadata | dict[str, Any] | None = None,
) -> dict[str, Any]:
    """Correlate a full Step 63 packet evidence payload against ONOS state (Step 64)."""
    if not isinstance(evidence, dict):
        raise EvidenceCorrelationError("Evidence payload must be a JSON object (dict)")

    observations = evidence.get("observations")
    if not isinstance(observations, list) or not observations:
        raise EvidenceCorrelationError("Evidence payload must contain a non-empty 'observations' list")

    ports_list = ports if isinstance(ports, list) else []
    hosts_list = hosts if isinstance(hosts, list) else []
    flows_list = flows if isinstance(flows, list) else []

    ports_by_device: dict[str, dict[str, dict[str, Any]]] = {device_id: {}}
    for port_entry in ports_list:
        if not isinstance(port_entry, dict):
            continue
        p_num = str(port_entry.get("port") or "").strip()
        p_dev = str(port_entry.get("element") or device_id).strip()
        if p_num:
            ports_by_device.setdefault(p_dev, {})[p_num] = port_entry

    # Index hosts across all referenced devices
    referenced_devices = {device_id}
    for obs in observations:
        if isinstance(obs, dict) and obs.get("switch_id"):
            referenced_devices.add(str(obs["switch_id"]).strip())

    hosts_by_device_port: dict[str, dict[str, list[dict[str, Any]]]] = {}
    for dev in referenced_devices:
        hosts_by_device_port[dev] = index_hosts_by_port(hosts_list, dev)

    ip_source_flow_count = count_ip_source_flows(flows_list)
```

```
    time_verification = verify_capture_time_window(
        evidence,
        capture_metadata=capture_metadata,
        alerts=alerts,
    )

    correlation_results: list[dict[str, Any]] = []
    for obs in observations:
        if not isinstance(obs, dict):
            obs = {}
        result = correlate_single_observation(
            obs,
            devices=devices,
            ports_by_device=ports_by_device,
            hosts_by_device_port=hosts_by_device_port,
            ip_source_flow_count=ip_source_flow_count,
            time_verification=time_verification,
        )
        correlation_results.append(result.to_dict())

    all_corroborated = all(r["topology_corroborated"] for r in correlation_results)
    any_attacker_confirmed = any(r["attacker_confirmed"] for r in
correlation_results)
    overall_decision = (
        PolicyDecisionType.PROPOSE_BLOCK.value
        if any_attacker_confirmed
        else PolicyDecisionType.OBSERVE.value
    )

    return {
        "schema_version": 2,
        "step": "STEP_64_ONOS_HOST_EVIDENCE_CORRELATION",
        "evidence_type": "onos_host_correlated_packet_observation",
        "device_id": device_id,
        "total_packets": evidence.get("total_packets", 0),
        "first_packet_clock": evidence.get("first_packet_clock"),
        "last_packet_clock": evidence.get("last_packet_clock"),
        "time_verification": time_verification,
        "topology_snapshot_summary": {
```

```
            "onos_device_count": len(devices) if isinstance(devices, list) else 1,
            "onos_port_count": len([p for p in ports_list if isinstance(p, dict) and
str(p.get("port")) != "local"]),
            "onos_host_count": len(hosts_list),
            "onos_flow_count": len(flows_list),
            "ip_source_selector_flows": ip_source_flow_count,
        },
        "correlated_observations": correlation_results,
        "security_assessment": {
            "source_ip_observed": True,
            "topology_corroborated": all_corroborated,
            "attacker_confirmed": any_attacker_confirmed,
            "historical_alert_correlation_verified":
time_verification["historical_alert_correlation_verified"],
            "decision": overall_decision,
            "automatic_block_allowed": False,
        },
    }
```

PH? L?C F. TO?N V?N M? NGU?N PH?N T?CH G?I TIN B?T T?I C?NG V?O (STEP 63)

T?p: backend/defense/source_ip_evidence_analyzer.py

```
import argparse
import hashlib
import ipaddress
import json
import re
import sys
from collections import Counter
from pathlib import Path


PATTERN = re.compile(
    r'^(?P<clock>\d{2}:\d{2}:\d{2}\.\d+)\s+'
    r'(?P<interface>s1-eth(?P<port>\d+))\s+\S+\s+'
    r'ifindex\s+\d+\s+'
```

```
    r'(?P<mac>(?:[0-9a-f]{2}:){5}[0-9a-f]{2})\s+'
    r'ethertype\s+IPv4\b.*?:\s+'
    r'(?P<src>(?:\d{1,3}\.){3}\d{1,3})\s+>\s+'
    r'(?P<dst>(?:\d{1,3}\.){3}\d{1,3}):\s+'
    r'ICMP\s+echo\s+(?P<kind>request|reply)\b',
    re.IGNORECASE,
)


def read_capture(path):
    data = path.read_bytes()

    # Windows PowerShell Tee-Object may write UTF-16LE.
    if data.startswith((b"\xff\xfe", b"\xfe\xff")):
        return data.decode("utf-16")

    if b"\x00" in data[:100]:
        return data.decode("utf-16-le")

    return data.decode("utf-8-sig")


def analyze(text, capture_metadata=None):
    records = []

    for line in text.splitlines():
        if not (
            re.search(r"s1-eth[12]\b", line)
            and "ethertype IPv4" in line
        ):
            continue

        match = PATTERN.search(line.strip())

        if not match:
            raise ValueError(
                "Unsupported switch IPv4 packet: " + line[:160]
            )
```

```
    packet = match.groupdict()

    ipaddress.IPv4Address(packet["src"])
    ipaddress.IPv4Address(packet["dst"])

    records.append(packet)

if not records:
    raise ValueError("No supported switch ICMP packets found")

counts = Counter(
    (
        packet["port"],
        packet["interface"],
        packet["mac"].lower(),
        packet["src"],
        packet["dst"],
        packet["kind"].lower(),
    )
    for packet in records
)

observations = []

for key, count in sorted(counts.items()):
    port, interface, mac, src, dst, kind = key

    observations.append({
        "switch_id": "of:0000000000000001",
        "ingress_port": port,
        "interface": interface,
        "observed_source_mac": mac,
        "observed_source_ip": src,
        "observed_destination_ip": dst,
        "protocol": "ICMP",
        "icmp_type": kind,
        "packet_count": count,
    })
```

```
    has_meta_window = bool(
        isinstance(capture_metadata, dict)
        and capture_metadata.get("capture_started_utc")
        and capture_metadata.get("capture_finished_utc")
    )

    payload = {
        "schema_version": 1,
        "evidence_type": "switch_ingress_packet_observation",
        "total_packets": len(records),
        "first_packet_clock": records[0]["clock"],
        "last_packet_clock": records[-1]["clock"],
        "capture_clock_has_date": has_meta_window,
        "capture_clock_has_timezone": has_meta_window,
        "observations": observations,
        "security_assessment": {
            "source_ip_observed": True,
            "attacker_confirmed": False,
            "historical_alert_correlation_verified": False,
            "decision": "OBSERVE",
            "automatic_block_allowed": False,
        },
    }
    if isinstance(capture_metadata, dict):
        payload["capture_metadata"] = dict(capture_metadata)
    return payload


def self_test():
    sample = "\n".join([
        "18:46:47.909111 s1-eth1 P ifindex 7 "
        "00:00:00:00:00:01 ethertype IPv4 (0x0800), "
        "length 104: 10.0.0.1 > 10.0.0.2: "
        "ICMP echo request, id 29154, seq 1, length 64",

        "18:46:47.910259 s1-eth2 P ifindex 8 "
        "00:00:00:00:00:02 ethertype IPv4 (0x0800), "
        "length 104: 10.0.0.2 > 10.0.0.1: "
        "ICMP echo reply, id 29154, seq 1, length 64",
```

```
    ])

    result = analyze(sample)

    assert result["total_packets"] == 2
    assert len(result["observations"]) == 2
    assert result["observations"][0]["observed_source_ip"] == "10.0.0.1"
    assert result["observations"][1]["observed_source_ip"] == "10.0.0.2"
    assert result["security_assessment"]["automatic_block_allowed"] is False

    meta_result = analyze(
        sample,
        capture_metadata={
            "capture_started_utc": "2026-11-01T18:46:45Z",
            "capture_finished_utc": "2026-11-01T18:46:55Z",
            "switch_id": "of:0000000000000001",
            "interfaces": ["s1-eth1", "s1-eth2"],
        },
    )
    assert meta_result["capture_clock_has_date"] is True
    assert meta_result["capture_clock_has_timezone"] is True
    assert meta_result["security_assessment"]["automatic_block_allowed"] is False

    print("SELF TEST PASS - Packet parser")
    print("SELF TEST PASS - Source IP and MAC")
    print("SELF TEST PASS - Capture UTC metadata support")
    print("SELF TEST PASS - No automatic blocking")
    print("SELF TEST PASS - No external requests")


def main():
    parser = argparse.ArgumentParser()

    parser.add_argument("--input")
    parser.add_argument("--started-utc")
    parser.add_argument("--finished-utc")
    parser.add_argument("--self-test", action="store_true")

    args = parser.parse_args()
```

```
if args.self_test:
    self_test()
    return

if not args.input:
    parser.error("--input is required")

source = Path(args.input)

if not source.is_file():
    raise FileNotFoundError(source)

capture_metadata = None
if args.started_utc and args.finished_utc:
    capture_metadata = {
        "capture_started_utc": args.started_utc,
        "capture_finished_utc": args.finished_utc,
        "capture_sha256": hashlib.sha256(source.read_bytes()).hexdigest(),
        "switch_id": "of:0000000000000001",
        "interfaces": ["s1-eth1", "s1-eth2"],
    }

result = analyze(read_capture(source), capture_metadata=capture_metadata)

output = source.with_suffix(".json")

# Exclusive creation prevents overwriting earlier evidence.
with output.open("x", encoding="utf-8") as file:
    json.dump(result, file, ensure_ascii=False, indent=2)
    file.write("\n")

print("=== SOURCE IP EVIDENCE ANALYSIS ===")
print("Input:", source)
print("Parsed packets:", result["total_packets"])

for item in result["observations"]:
    print(
        "Port={ingress_port} "
```

```
            "MAC={observed_source_mac} "
            "SRC={observed_source_ip} "
            "DST={observed_destination_ip} "
            "ICMP={icmp_type} "
            "Packets={packet_count}".format(**item)
        )

    print("\nAttacker confirmed: False")
    print("Defense decision: OBSERVE")
    print("Automatic blocking: DISABLED")
    print("Historical alert correlation: NOT VERIFIED")
    print("Output JSON:", output)
    print("\nSUCCESS - Source IP Evidence Analyzer completed")


if __name__ == "__main__":
    try:
        main()
    except Exception as exc:
        print("ERROR:", exc, file=sys.stderr)
        sys.exit(1)
```

PH? L?C G. TO?N V?N M? NGU?N ??I CHI?U TOPOLOGY ONOS TR?C TI?P (STEP 64)

T?p: backend/defense/onos_host_evidence_correlator.py

```
"""Step 64: ONOS Host Evidence Correlator CLI and Library Module.

Correlates Step 63 packet observations (ingress_icmp_*.json) against ONOS
topology state (devices, ports, hosts,
```

```
lows) and optional Elasticsearch
alert windows, enforcing strict timestamp and source attribution safety rules.
"""

from __future__ import annotations

import argparse
import base64
import json
import os
import sys
from pathlib import Path
from typing import Any
from urllib.request import Request, urlopen

BACKEND_ROOT = Path(__file__).resolve().parents[1]
if str(BACKEND_ROOT) not in sys.path:
    sys.path.insert(0, str(BACKEND_ROOT))

from sdnflowguard.domain.models import CorrelationStatus, PolicyDecisionType  #
noqa: E402
from sdnflowguard.domain.services.evidence_correlation import (  # noqa: E402
    compute_file_sha256,
    correlate_evidence_with_onos,
)

ONOS_URL = "http://127.0.0.1:8181/onos/v1"
DEVICE_ID = "of:0000000000000001"
DEFAULT_EVIDENCE_PATH = (
    Path(__file__).resolve().parent / "evidence" /
"ingress_icmp_20261101_014645.json"
)

DEFAULT_LAB_TOPOLOGY = {
    "devices": [
        {
            "id": DEVICE_ID,
            "available": True,
            "role": "MASTER",
```

```
            "type": "SWITCH",
        }
    ],
    "ports": [
        {"element": DEVICE_ID, "port": "local", "isEnabled": True},
        {"element": DEVICE_ID, "port": "1", "isEnabled": True},
        {"element": DEVICE_ID, "port": "2", "isEnabled": True},
    ],
    "hosts": [
        {
            "id": "00:00:00:00:00:01/None",
            "mac": "00:00:00:00:00:01",
            "vlan": "None",
            "ipAddresses": ["10.0.0.1"],
            "locations": [{"elementId": DEVICE_ID, "port": "1"}],
        },
        {
            "id": "00:00:00:00:00:02/None",
            "mac": "00:00:00:00:00:02",
            "vlan": "None",
            "ipAddresses": ["10.0.0.2"],
            "locations": [{"elementId": DEVICE_ID, "port": "2"}],
        },
    ],
    "flows": [
        {
            "id": "10001",
            "appId": "org.onosproject.core",
            "selector": {"criteria": [{"type": "ETH_TYPE", "ethType": "0x800"}]},
        }
    ],
}


def onos_get(path: str) -> dict[str, Any]:
    """Perform a read-only GET request against the ONOS REST API."""
    user = os.environ.get("SDNFLOW_ONOS_USER")
    password = os.environ.get("SDNFLOW_ONOS_PASSWORD")
    if not user or not password:
```

```
        raise RuntimeError("ONOS credentials (SDNFLOW_ONOS_USER /
SDNFLOW_ONOS_PASSWORD) are not configured")

    credentials = base64.b64encode(f"{user}:{password}".encode("utf-
8")).decode("ascii")
    req = Request(
        f"{ONOS_URL}/{path}",
        headers={
            "Authorization": f"Basic {credentials}",
            "Accept": "application/json",
        },
        method="GET",
    )
    with urlopen(req, timeout=15) as resp:
        return json.load(resp)


def load_json_file(path: Path) -> dict[str, Any]:
    """Read and parse a UTF-8 JSON file."""
    with path.open("r", encoding="utf-8-sig") as handle:
        data = json.load(handle)
    if not isinstance(data, dict):
        raise ValueError(f"Expected JSON object in {path}")
    return data


def correlate_step63_evidence_file(
    evidence_path: Path | None = None,
    *,
    topology: dict[str, Any] | None = None,
    capture_metadata: dict[str, Any] | None = None,
    alerts: list[dict[str, Any]] | None = None,
) -> dict[str, Any]:
    """Load Step 63 evidence file and correlate with ONOS topology snapshot."""
    target_path = evidence_path or DEFAULT_EVIDENCE_PATH
    if not target_path.is_file():
        raise FileNotFoundError(f"Step 63 evidence file not found: {target_path}")

    evidence_data = load_json_file(target_path)
```

```
    topo = topology if isinstance(topology, dict) else DEFAULT_LAB_TOPOLOGY

    report = correlate_evidence_with_onos(
        evidence_data,
        device_id=DEVICE_ID,
        devices=topo.get("devices"),
        ports=topo.get("ports"),
        hosts=topo.get("hosts"),
        flows=topo.get("flows"),
        alerts=alerts,
        capture_metadata=capture_metadata,
    )
    report["evidence_source_file"] = target_path.name
    report["evidence_source_sha256"] = compute_file_sha256(target_path)
    report["topology_source"] = "LIVE_ONOS" if topology is not None else
"REFERENCE_LAB_TOPOLOGY_FIXTURE"
    return report


def self_test() -> None:
    """Execute offline verification covering all Step 64 correlation edge cases."""
    sample_evidence = {
        "schema_version": 1,
        "evidence_type": "switch_ingress_packet_observation",
        "total_packets": 24,
        "first_packet_clock": "18:46:47.909111",
        "last_packet_clock": "18:46:50.146161",
        "capture_clock_has_date": False,
        "capture_clock_has_timezone": False,
        "observations": [
            {
                "switch_id": DEVICE_ID,
                "ingress_port": "1",
                "interface": "s1-eth1",
                "observed_source_mac": "00:00:00:00:00:01",
                "observed_source_ip": "10.0.0.1",
                "observed_destination_ip": "10.0.0.2",
                "protocol": "ICMP",
                "icmp_type": "request",
```

```
                "packet_count": 12,
            },
            {
                "switch_id": DEVICE_ID,
                "ingress_port": "2",
                "interface": "s1-eth2",
                "observed_source_mac": "00:00:00:00:00:02",
                "observed_source_ip": "10.0.0.2",
                "observed_destination_ip": "10.0.0.1",
                "protocol": "ICMP",
                "icmp_type": "reply",
                "packet_count": 12,
            },
        ],
    }

    # Case 1: Single host corroborated, but no date/timezone -> OBSERVE, no block
    res1 = correlate_evidence_with_onos(
        sample_evidence,
        device_id=DEVICE_ID,
        devices=DEFAULT_LAB_TOPOLOGY["devices"],
        ports=DEFAULT_LAB_TOPOLOGY["ports"],
        hosts=DEFAULT_LAB_TOPOLOGY["hosts"],
        flows=DEFAULT_LAB_TOPOLOGY["flows"],
    )
    assert res1["security_assessment"]["topology_corroborated"] is True
    assert res1["security_assessment"]["historical_alert_correlation_verified"] is
False
    assert res1["security_assessment"]["attacker_confirmed"] is False
    assert res1["security_assessment"]["decision"] ==
PolicyDecisionType.OBSERVE.value
    assert res1["security_assessment"]["automatic_block_allowed"] is False
    assert (
        res1["correlated_observations"][0]["correlation_status"]
        == CorrelationStatus.CORROBORATED_HOST_OBSERVATION.value
    )

    # Case 2: Multi-host on same port -> AMBIGUOUS_MULTI_HOST
    multi_hosts = [
```

```
    {
        "mac": "00:00:00:00:00:01",
        "ipAddresses": ["10.0.0.1"],
        "locations": [{"elementId": DEVICE_ID, "port": "1"}],
    },
    {
        "mac": "00:00:00:00:00:09",
        "ipAddresses": ["10.0.0.9"],
        "locations": [{"elementId": DEVICE_ID, "port": "1"}],
    },
]
res2 = correlate_evidence_with_onos(
    sample_evidence,
    device_id=DEVICE_ID,
    ports=DEFAULT_LAB_TOPOLOGY["ports"],
    hosts=multi_hosts,
    flows=[],
)
assert (
    res2["correlated_observations"][0]["correlation_status"]
    == CorrelationStatus.AMBIGUOUS_MULTI_HOST.value
)
assert res2["correlated_observations"][0]["topology_corroborated"] is False

# Case 3: MAC/IP mismatch -> HOST_IDENTITY_MISMATCH
spoof_hosts = [
    {
        "mac": "00:00:00:00:00:aa",
        "ipAddresses": ["10.0.0.99"],
        "locations": [{"elementId": DEVICE_ID, "port": "1"}],
    }
]
res3 = correlate_evidence_with_onos(
    sample_evidence,
    device_id=DEVICE_ID,
    ports=DEFAULT_LAB_TOPOLOGY["ports"],
    hosts=spoof_hosts,
    flows=[],
)
```

```
assert (
    res3["correlated_observations"][0]["correlation_status"]
    == CorrelationStatus.HOST_IDENTITY_MISMATCH.value
)

# Case 4: Disabled port -> PORT_UNAVAILABLE_OR_DISABLED
disabled_ports = [
    {"element": DEVICE_ID, "port": "1", "isEnabled": False},
    {"element": DEVICE_ID, "port": "2", "isEnabled": True},
]
res4 = correlate_evidence_with_onos(
    sample_evidence,
    device_id=DEVICE_ID,
    ports=disabled_ports,
    hosts=DEFAULT_LAB_TOPOLOGY["hosts"],
    flows=[],
)
assert (
    res4["correlated_observations"][0]["correlation_status"]
    == CorrelationStatus.PORT_UNAVAILABLE_OR_DISABLED.value
)

# Case 5: Invalid observation data -> INVALID_EVIDENCE_DATA
bad_evidence = {
    "observations": [
        {
            "switch_id": DEVICE_ID,
            "ingress_port": "1",
            "observed_source_mac": "not-a-mac",
            "observed_source_ip": "999.999.999.999",
            "packet_count": -1,
        }
    ]
}
res5 = correlate_evidence_with_onos(
    bad_evidence,
    device_id=DEVICE_ID,
    ports=DEFAULT_LAB_TOPOLOGY["ports"],
    hosts=DEFAULT_LAB_TOPOLOGY["hosts"],
```

```
        flows=[],
    )
    assert (
        res5["correlated_observations"][0]["correlation_status"]
        == CorrelationStatus.INVALID_EVIDENCE_DATA.value
    )

    # Case 6: Real Step 63 evidence file verification if present
    if DEFAULT_EVIDENCE_PATH.is_file():
        real_res = correlate_step63_evidence_file(DEFAULT_EVIDENCE_PATH)
        assert real_res["total_packets"] == 24
        assert real_res["security_assessment"]["decision"] == "OBSERVE"
        assert real_res["security_assessment"]["automatic_block_allowed"] is False

    print("SELF TEST PASS - Step 64 single-host topology corroboration")
    print("SELF TEST PASS - Step 64 multi-host port ambiguity detection")
    print("SELF TEST PASS - Step 64 IP/MAC identity mismatch detection")
    print("SELF TEST PASS - Step 64 disabled/missing port & device handling")
    print("SELF TEST PASS - Step 64 invalid evidence rejection")
    print("SELF TEST PASS - Step 64 timestamp & test_only safety guardrails")


def main() -> None:
    parser = argparse.ArgumentParser(
        description="Step 64: Correlate packet evidence against ONOS host topology."
    )
    parser.add_argument(
        "--evidence",
        default=str(DEFAULT_EVIDENCE_PATH),
        help="Path to Step 63 packet observation JSON file.",
    )
    parser.add_argument(
        "--live-onos",
        action="store_true",
        help="Query live ONOS REST API instead of reference lab topology fixture.",
    )
    parser.add_argument(
        "--self-test",
        action="store_true",
```

```
    help="Run built-in Step 64 verification tests with zero network requests.",
)
args = parser.parse_args()

if args.self_test:
    self_test()
    return

evidence_path = Path(args.evidence)
topology = None
if args.live_onos:
    devices_res = onos_get("devices")
    ports_res = onos_get(f"devices/{DEVICE_ID}/ports")
    hosts_res = onos_get("hosts")
    flows_res = onos_get(f"flows/{DEVICE_ID}")
    topology = {
        "devices": devices_res.get("devices") or [],
        "ports": ports_res.get("ports") or [],
        "hosts": hosts_res.get("hosts") or [],
        "flows": flows_res.get("flows") or [],
    }

report = correlate_step63_evidence_file(evidence_path, topology=topology)
print("=== STEP 64: ONOS HOST EVIDENCE CORRELATION ===")
print("Evidence file:", report["evidence_source_file"])
print("Evidence SHA-256:", report["evidence_source_sha256"])
print("Topology source:", report["topology_source"])
print("Total packets:", report["total_packets"])
print(
    "Historical alert correlation verified:",
    report["time_verification"]["historical_alert_correlation_verified"],
)
print("Time verification reason:", report["time_verification"]["reason"])
for item in report["correlated_observations"]:
    print(
        f"Port={item['ingress_port']} MAC={item['observed_source_mac']} "
        f"IP={item['observed_source_ip']} Status={item['correlation_status']} "
        f"Decision={item['recommended_decision']}"
    )
```

```
    print("Overall Decision:", report["security_assessment"]["decision"])
    print("Automatic Block Allowed:",
report["security_assessment"]["automatic_block_allowed"])
    print("\nSUCCESS - Step 64 ONOS Host Evidence Correlation completed")


if __name__ == "__main__":
    try:
        main()
    except Exception as exc:
        print(f"ERROR: {exc}", file=sys.stderr)
        sys.exit(1)
```

PH? L?C H. TO?N V?N C?U H?NH PIPELINE LOGSTASH V? DOCKER COMPOSE

T?p: infrastructure/logstash/pipeline/01-sdnflowguard.conf

```
input {
  http {
    host => "0.0.0.0"
    port => 8080
    codec => json
  }
}

filter {
  mutate {
    add_field => {
      "project" => "SDNFlowGuard"
    }
  }
}

output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "sdnflowguard-events-%{+YYYY.MM.dd}"
```

```
    ilm_enabled => false
  }
}
```

T?p: infrastructure/compose.yaml

```
name: sdnflowguard-netsec

services:
  onos:
    image: onosproject/onos:2.7-latest
    container_name: sdnflowguard-onos
    restart: unless-stopped
    ports:
      - "127.0.0.1:8181:8181"
      - "127.0.0.1:6653:6653"

  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.15.1
    container_name: sdnflowguard-elasticsearch
    restart: unless-stopped
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - xpack.ml.enabled=false
      - ES_JAVA_OPTS=-Xms512m -Xmx512m
    mem_limit: 2g
    ports:
      - "127.0.0.1:9200:9200"
    volumes:
      - esdata:/usr/share/elasticsearch/data

  logstash:
    image: docker.elastic.co/logstash/logstash:8.15.1
    container_name: sdnflowguard-logstash
    restart: unless-stopped
    environment:
      LS_JAVA_OPTS: "-Xms384m -Xmx384m -XX:MaxDirectMemorySize=128m"
      XPACK_MONITORING_ENABLED: "false"
      PIPELINE_WORKERS: "1"
```

```
    mem_limit: 1536m
    ports:
      - "127.0.0.1:18080:8080"
    volumes:
      - ./logstash/pipeline:/usr/share/logstash/pipeline:ro
    depends_on:
      - elasticsearch

  kibana:
    image: docker.elastic.co/kibana/kibana:8.15.1
    container_name: sdnflowguard-kibana
    profiles:
      - ui
    restart: "no"
    environment:
      SERVER_NAME: "sdnflowguard-kibana"
      ELASTICSEARCH_HOSTS: '["http://elasticsearch:9200"]'
    mem_limit: 2g
    ports:
      - "127.0.0.1:5601:5601"
    depends_on:
      - elasticsearch

volumes:
  esdata:
    external: true
    name: sdnflowguard-elasticsearch-data
```